FACULDADE DE ENGENHARIA DA UNIVERSIDADE DO PORTO

Enhancing Centralized Multi-Robot Path Planning and Coordination

Afonso Tomás de Magalhães Mateus

Mestrado em Engenharia Eletrotécnica e de Computadores

Supervisor: Prof. Pedro Luís Cerqueira Gomes da Costa

Second Supervisor: Prof. Diogo Miguel Rodrigues Matos

May 19, 2026

Abstract

This dissertation addresses the problem of centralized coordination of multiple autonomous mobile robots operating in shared environments. As robotic fleets become increasingly prevalent in industrial, logistics, and service applications, the ability to compute coordinated, collision-free trajectories under real-world constraints becomes a critical requirement. This challenge is commonly formalized as the Multi-Agent Path Finding (MAPF) problem, which seeks to determine feasible paths for multiple agents from given start locations to specified goal locations while avoiding conflicts over time.

MAPF is computationally hard in its general form, which has motivated the development of practical planning approaches that trade optimality for scalability and real-time performance. In centralized fleet coordination systems, priority-based planners are often adopted due to their predictable behavior and computational efficiency. In particular, the centralized coordination framework developed at the Institute for Systems and Computer Engineering, Technology and Science (INESC TEC), within the Centre for Robotics in Industry and Intelligent Systems (CRIIS) and the Industry and Innovation Laboratory (iiLab), relies on a sequential Time-Enhanced A* (TEA*) planner, which incorporates temporal reasoning and space–time reservations to ensure conflict-free execution. While well suited for real-world deployment, this sequential structure introduces a significant limitation: planning outcomes are highly sensitive to the chosen robot priority ordering, and exhaustive exploration of all orderings is infeasible due to factorial growth with fleet size.

The main motivation of this work is to reduce the sensitivity of sequential TEA*-based planning to robot ordering without compromising the existing centralized architecture. Rather than replacing the underlying planner, this dissertation explores the use of parallel computation to improve solution robustness within a fixed planning time budget. The proposed approach executes multiple TEA* instances concurrently, each operating on a different priority ordering of the same MAPF instance, and selects the most suitable solution among those evaluated.

The solution builds upon a centralized planning framework in which a Supervisor issues planning requests composed of a set of agents and their associated start and goal vertices. Within the Path Planner, heuristic functions generate diverse priority orderings that are processed independently in parallel threads. Although TEA* remains sequential within each thread, the overall planning process benefits from parallelism at the system level. Alternative execution strategies are also discussed, including early solution selection based on partial thread completion or fixed time budgets, enabling a tunable trade-off between planning latency and exploration depth.

Finally, the document outlines a structured work plan guiding the implementation and experimental evaluation of the proposed solution. Overall, this dissertation aims to contribute a practical and extensible approach for improving centralized multi-robot coordination by leveraging parallel planning to enhance robustness and solution quality in TEA*-based fleet coordination systems.

Contents

List of Figures

List of Acronyms

APF	*Artificial Potential Field*
CCBS	*Continuous-Time Conflict-Based Search*
CBS	*Conflict-Based Search*
CBSH	*Heuristic-Guided Conflict-Based Search*
CBSH-RTC	*Real-Time Conflict-Based Search with Heuristics*
CRIIS	*Centre for Robotics in Industry and Intelligent Systems*
ECBS	*Enhanced Conflict-Based Search*
INESC TEC	*Institute for Systems and Computer Engineering, Technology and Science*
MAPF	*Multi-Agent Path Finding*
NP-hard	*Nondeterministic Polynomial-time Hard*
ROS	*Robot Operating System*
SIPP	*Safe Interval Path Planning*
TEA*	*Time-Enhanced A-Star*
iiLab	*Industry and Innovation Laboratory*
DoF	*Degrees of Freedom*

Chapter 1

Introduction

1.1 Context

The increasing adoption of autonomous mobile robots in industrial, logistics, and service environments has intensified the need for effective coordination of multiple agents operating in shared spaces. In such scenarios, robots must navigate efficiently while avoiding collisions and mutual interference, often under strict temporal and operational constraints. As fleet sizes grow, coordination becomes a key challenge that directly impacts system performance, scalability, and reliability.

This problem is commonly formalized as the MAPF problem, which focuses on computing collision-free trajectories for multiple agents moving from initial to goal states in a shared environment [1]. By explicitly modeling agent interactions over time, MAPF has become a central abstraction for multi-robot coordination in both centralized and decentralized planning frameworks.

MAPF is computationally challenging, with optimal variants being NP-hard (Nondeterministic Polynomial-time hard) in the general case [2]. Consequently, a wide spectrum of solution approaches has been proposed, ranging from optimal but computationally demanding methods to heuristic and approximate techniques that favor scalability and real-time performance [3]. Within this landscape, MAPF serves as a foundational framework for studying coordination, conflict resolution, and scalability in multi-robot systems.

1.2 Motivation and Problem

This dissertation is developed in the context of research on centralized multi-robot fleet coordination conducted at INESC TEC, namely at CRIIS and the iiLab. Within this context, a centralized fleet coordination framework has been proposed to address multi-robot navigation under realistic operational constraints, relying on a shared graph-based environment representation and a sequential TEA* planning component [4]–[6].

In this framework, robots are planned sequentially according to a predefined priority order, which ensures conflict avoidance by construction but makes the resulting solution highly sensitive

to robot ordering. Different priority permutations may lead to substantially different outcomes, particularly in constrained environments with shared resources. As the number of possible orderings grows factorially with fleet size, exhaustive exploration becomes computationally infeasible.

These limitations motivate the investigation of parallel planning strategies. By exploiting multithreading to evaluate multiple priority orderings concurrently, the impact of ordering sensitivity can be reduced while preserving compatibility with the existing TEA*-based coordination framework.

1.3 Objectives

The main objective of this dissertation is to improve the robustness and effectiveness of a centralized TEA*-based multi-robot planner by reducing its sensitivity to robot prioritization through parallel planning. To this end, the impact of robot ordering on the feasibility and performance of sequential TEA*-based planning is analyzed, and a multithreaded approach is designed in which multiple TEA* instances are executed in parallel, each considering a different priority ordering. Heuristic strategies are developed to systematically generate and distribute priority orderings across parallel threads, and the proposed approach is integrated into an existing centralized fleet coordination framework. Finally, the solution is experimentally evaluated in representative multi-robot scenarios, assessing solution quality and computational performance.

1.4 Potential Contributions

This dissertation builds upon the centralized fleet coordination system described by Matos et al. [4] and proposes a multithreaded extension of a sequential TEA*-based planner that enables the parallel exploration of multiple robot priority orderings. The primary expected contribution is the ability to obtain higher-quality coordination solutions within the same planning time, without modifying the core architecture of the existing system.

By evaluating a broader set of priority orderings, the proposed approach aims to improve robustness and performance in constrained multi-robot scenarios. Additionally, this work provides empirical insights into the effects of parallelism in priority-based planning, including comparative evaluation against alternative approaches, contributing to the design of future centralized fleet coordination systems.

1.5 Document Structure

This document is organized into five chapters. Chapter 1 introduces the problem and objectives. Chapter 2 presents the background and related work. Chapter 3 describes algorithmic approaches to MAPF and the baseline fleet coordination framework. Chapter 4 presents the preliminary solution and work plan. Finally, Chapter 8 concludes the document and outlines its key conclusions.

Chapter 2

Background and Fundamental Aspects

This chapter presents the fundamental concepts and background knowledge required to contextualize the research developed in this dissertation. Section 2.1 and Section 2.2 introduces a formal definition of the MAPF problem and its representation, respectively. Section 2.3 explore classical graph search algorithms that serve as building blocks for many MAPF methods. Finally, Section 2.4 introduces foundational notions of parallel and concurrent computing.

2.1 MAPF Problem: Formal Definition

The MAPF problem concerns the computation of collision-free trajectories for a group of cooperative agents that must move from their initial states to their target states while operating in a shared environment. Each agent must reach its goal without colliding with other agents, while a global performance criterion is optimized. This general problem formulation follows the, widely adopted, conceptual definition of MAPF presented by Stern et al. [1]. While that work instantiates the problem on graph-based environments, the underlying definition is independent of the specific representation and can be abstracted to more general state spaces.

From a formal perspective, let there be a set of m agents $A = \{a_1, a_2, \ldots, a_m\}$. Each agent $a_i \in A$ is associated with an initial state s_i and a goal state g_i.

A solution for an agent a_i is defined as a trajectory P_i, which specifies the evolution of the agent's state over time. In discrete formulations, a trajectory can be represented as a sequence of states

$$P_i = \langle p_i^0, p_i^1, \ldots, p_i^T \rangle \tag{2.1}$$

where $p_i^0 = s_i$ and $p_i^T = g_i$. The explicit consideration of time is essential in MAPF, as interactions and conflicts between agents arise from the simultaneous execution of their trajectories in a shared space-time domain [1].

A valid MAPF solution must ensure that no two agents are in conflict during their motion, where conflicts arise when agents attempt to occupy incompatible states in space and time. The most common type of conflict is the position conflict, which occurs when two agents occupy the same state at the same time step, i.e., $p_i^t = p_j^t$ for $i \neq j$. Other types of conflicts may arise depending on

the motion model and representation adopted, but all correspond to violations of mutual feasibility constraints among agents [1].

The objective of MAPF is to compute a set of conflict-free trajectories $\{P_1, P_2, \ldots, P_m\}$ that minimize a global cost function C. This cost function is defined over the joint set of agent trajectories and encodes the chosen optimization criterion. Common objectives include minimizing the time at which the last agent reaches its goal (makespan) or minimizing the sum of individual agents' traversal costs, as commonly considered in the literature. Formally:

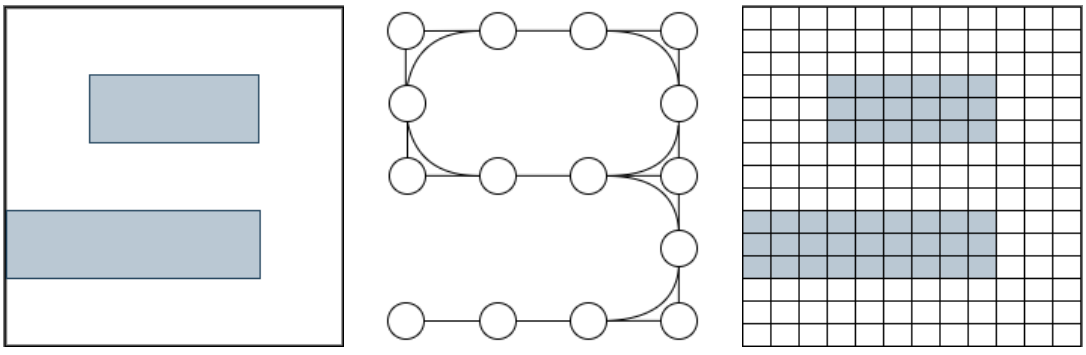
Find $\{P_1, P_2, \ldots, P_m\}$ **such that** all trajectories are conflict-free and $C(\{P_i\})$ is minimized.

The MAPF problem is combinatorial in nature and is NP-hard in the general case, as formally established in earlier works [2], [7] and confirmed in recent complexity analyses [8], [9]. This inherent complexity motivates the development of planning algorithms that aim to balance solution quality with computational efficiency.

2.2 MAPF Problem: Representation

Although the MAPF problem can be defined independently of the underlying environment representation, practical solutions require instantiating the problem on a concrete state space. Different environment representations have been adopted in the MAPF literature, primarily differing in how space and time are modeled and in the resulting level of abstraction.

Common approaches include discrete representations, such as grids and graphs, where agents occupy a finite set of states and move between neighboring states in discrete time steps, as well as formulations based on continuous space and time, where agent motion is described by continuous trajectories and conflicts arise from geometric interactions. Figure 2.1 illustrates how the same workspace can be modeled under different discrete representations.



(a) Workspace layout (b) Graph-based representation (c) Grid-based representation

Figure 2.1: Illustration of different discrete environment representations for the same workspace.

While continuous formulations offer higher modeling fidelity, discrete representations are more commonly adopted in classical MAPF settings due to their simplicity and structured abstraction. In the remainder of this section, the focus is placed on grid and graph-based representations, which provide a flexible and well-defined framework for modeling structured environments.

2.2.1 Grid-Based Representation

In grid-based MAPF formulations, the environment is discretized into a regular two or three-dimensional lattice of uniformly sized cells, each classified as free or occupied. Agents move between neighboring cells according to a fixed connectivity structure, yielding a simple and intuitive abstraction that has been widely adopted in early MAPF and multi-robot navigation studies [1], [10].

A key advantage of grid-based representations is their simplicity and direct correspondence to occupancy grids used in robotic perception. However, grid resolution introduces an inherent trade-off: coarse grids may fail to capture narrow passages or thin obstacles, while fine grids incur substantial memory and computational costs [10], [11]. In addition, the discrete nature of grid-based motion complicates the integration of continuous-time dynamics, often requiring post-processing to obtain executable trajectories [11].

For these reasons, many contemporary MAPF approaches favor graph-based representations, which generalize grids by allowing arbitrary topologies and connectivity patterns, making them a convenient abstraction for modeling structured environments in MAPF [1].

2.2.2 Graph-Based Representation

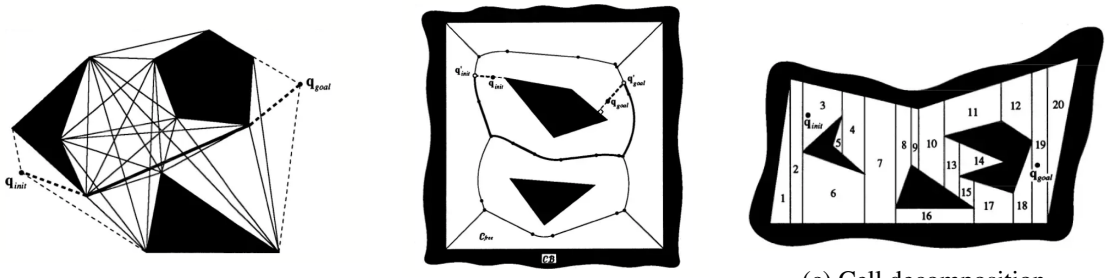
In graph-based formulations of the MAPF problem, the environment is modeled as a discrete graph that captures admissible agent states and their connectivity, following the modeling approach described by Stern et al. [1].

Formally, the environment is represented as a graph $G = (V, E)$, where each vertex $v \in V$ corresponds to a valid state that an agent may occupy, and each edge $(u, v) \in E$ encodes a feasible transition between states. The graph may be directed or undirected, depending on the characteristics of the environment. Under this representation, agent trajectories evolve in discrete time steps and are constrained to lie on the graph, such that each state p_i^t corresponds to a vertex in V and successive states satisfy $p_i^{t+1} = p_i^t \quad \text{or} \quad (p_i^t, p_i^{t+1}) \in E$ as commonly assumed in graph-based multi-robot and MAPF formulations [2].

2.2.2.1 Graph Construction Approaches

Several classical approaches have been proposed to construct graph-based representations of planning environments. These include roadmap-based approaches such as visibility graphs, which connect mutually visible obstacle vertices [12], Voronoi diagram-based representations that emphasize obstacle clearance [13], and cell decomposition techniques that subdivide the free space into regions whose adjacency defines a connectivity graph [14], [15]. Representative examples of

these constructions are illustrated in Fig. 2.2. However, in this dissertation, the focus is on MAPF formulations in which the underlying graph is assumed to be given, and agent coordination is performed on top of this discrete abstraction.



(a) Visibility graph

(b) Voronoi diagram

(c) Cell decomposition

Figure 2.2: Classical approaches for constructing graph-based representations [14].

2.3 Search and Planning Foundations

Many MAPF approaches rely on classical graph search techniques for computing individual agent paths and for addressing coordination constraints. Accordingly, this section reviews optimal graph search and planning foundations that underlie a wide range of MAPF methods and will be revisited when discussing multi-agent coordination strategies.

2.3.1 Search Problems on Graphs

A search problem on a graph consists of finding a sequence of transitions that leads from a given initial state to a goal state while satisfying feasibility constraints and optimizing a specified cost criterion [16].

Formally, a search problem is defined by a state space, an initial state, a set of goal states, a transition model that specifies the allowable moves between states, and a cost function that assigns a non-negative cost to each transition or path [16]. When the state space is represented explicitly as a graph, vertices correspond to states and edges encode feasible transitions, possibly associated with traversal costs.

A solution to a graph search problem is a path in the graph that connects the initial vertex to a goal vertex. Among all feasible solutions, particular interest is often placed on optimal solutions, defined as those that minimize the cumulative cost along the path. The shortest path problem in weighted graphs is a canonical instance of this formulation and has been extensively studied in both theoretical and applied contexts [17].

Search algorithms typically operate by incrementally exploring the graph starting from the initial state, expanding states according to a specific strategy and maintaining a frontier of candidate paths. Different search strategies define different orders in which states are expanded, which directly affects properties such as completeness, optimality, and computational efficiency [18].

2.3.1.1 Dijkstra's Algorithm

Dijkstra's algorithm is a classical graph search algorithm for solving the shortest path problem in weighted graphs with non-negative edge costs. Originally introduced by Dijkstra [19], it provides a systematic procedure for computing the minimum-cost path from a given initial vertex to all other reachable vertices in the graph.

The algorithm operates by incrementally expanding vertices in order of increasing accumulated path cost from the initial state. At each step, the vertex with the lowest known cost is selected for expansion, and its outgoing edges are relaxed to update the cost of reaching neighboring vertices. This process continues until the goal vertex is reached or all reachable vertices have been explored [17]. A key property of Dijkstra's algorithm is that, under the assumption of non-negative edge costs, once a vertex is expanded, the cost associated with it is guaranteed to be optimal. As a result, the algorithm is both complete and optimal for this class of graphs [17].

From a search perspective, Dijkstra's algorithm can be viewed as a best-first search strategy in which the expansion order is determined by the accumulated path cost from the initial vertex. This accumulated cost, commonly denoted by $g(n)$ for a vertex n, corresponds to the sum of the costs of the edges along the path from the initial vertex to n,

$$g(n) = \sum_{(u,v)\in P_n} c(u,v) \tag{2.2}$$

where P_n denotes the path to n and $c(u,v)$ is the cost of edge (u,v).

Regarding computational complexity, the cost of Dijkstra's algorithm depends on the data structure used to manage the frontier. When implemented with a binary heap priority queue and an adjacency-list graph representation, the algorithm has worst-case time complexity

$$O\big((|V|+|E|)\log|V|\big), \tag{2.3}$$

where $|V|$ is the number of vertices and $|E|$ is the number of edges in the graph. This bound results from inserting and updating vertices in the priority queue while relaxing the graph edges.

In MAPF settings, it is often used as a baseline within more complex coordination frameworks, serving as a reference point for evaluating the benefits more advanced search strategies.

2.3.1.2 A* Search Algorithm

A* is a heuristic-based graph search algorithm that extends Dijkstra's algorithm by incorporating problem-specific information to guide the search toward the goal more efficiently. Originally introduced by Hart, Nilsson, and Raphael [18], A* is designed to compute optimal paths while reducing the number of explored states when compared to uninformed search methods.

The algorithm evaluates each vertex using an objective function

$$f(n) = g(n) + h(n) \tag{2.4}$$

where $g(n)$ denotes the accumulated cost from the initial vertex to vertex n, as defined in Equation (2.2), and $h(n)$ is a heuristic estimate of the remaining cost from n to a goal vertex.

This algorithm maintains a frontier of candidate vertices ordered by increasing values of $f(n)$ and iteratively expands the vertex with the lowest estimated total cost. When the heuristic function is admissible, meaning that it never overestimates the true remaining cost to the goal, A* is guaranteed to be both complete and optimal [18], [20].

(a) Dijkstra (b) A*

Figure 2.3: Comparison of node expansion patterns produced by Dijkstra's algorithm and A*.

Figure 2.3 illustrates the resulting difference in exploration behavior between Dijkstra's algorithm and A*. The light blue marker denotes the initial state, the green marker denotes the goal state, and darker blue cells correspond to expanded nodes. The environment is modeled as a grid to facilitate visualization, and the Manhattan distance heuristic is used to guide the A* search. As can be observed, the heuristic effectively directs the search toward the goal, resulting in a focused expansion pattern, whereas Dijkstra's algorithm exhibits a nearly radial expansion that explores the state space more uniformly.

From a search perspective, Dijkstra's algorithm can be seen as a special case of A* in which the heuristic function is identically zero. Under this interpretation, A* generalizes Dijkstra by allowing heuristic guidance while preserving optimality guarantees under appropriate conditions [17].

The computational complexity of A* also depends on the implementation of the frontier and on the effectiveness of the heuristic function. In the worst case, when the heuristic provides no useful guidance, A* may explore the same search space as Dijkstra's algorithm. With a binary heap priority queue and an adjacency-list graph representation, its worst-case time complexity is therefore the same as in Equation 2.3.

In practice, an informative admissible heuristic can reduce the number of expanded vertices, but the worst-case bound remains the same.

2.4 Foundations of Parallel and Concurrent Computing

The MAPF problem involves the simultaneous evolution of multiple agents within a shared environment. As the number of agents increases, both the computational effort required to compute

individual paths and the complexity of coordinating their interactions grow rapidly. For this reason, many MAPF approaches rely on principles drawn from parallel and concurrent computing.

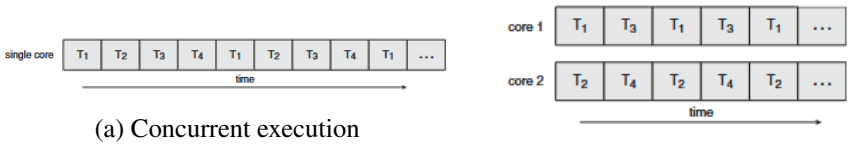
For this reason, concepts from parallel and concurrent computing are particularly relevant for addressing coordination challenges in MAPF.

2.4.1 Parallelism and Concurrency

Parallelism and concurrency are closely related but conceptually distinct notions that describe different aspects of task execution within a computational system.

In this context, parallelism refers to the simultaneous execution of multiple computational tasks, typically with the objective of improving performance or reducing overall execution time. It relies on the availability of multiple processing units, such as multi-core processors, and focuses on exploiting hardware resources to increase computational throughput [21].

Concurrency, in contrast, concerns the logical composition of multiple tasks that make progress overlapping periods of time, regardless of whether they are executed simultaneously in physical terms. These concepts are visually illustrated in Figure 2.4.



(a) Concurrent execution

(b) Parallel execution

Figure 2.4: Conceptual distinction between concurrency and parallelism [22].

2.4.2 Computational Models and Execution

Computational models provide an abstract description of how tasks are executed and how computational resources are organized during execution. These models are essential for reasoning about the structure, scalability, and coordination of computations involving multiple activities that progress over time.

2.4.2.1 Processes

A process represents an executing instance of a program together with its associated execution context. This context typically includes a private address space, program code, data, and operating system resources required for execution [23].

Processes provide strong isolation between executing programs, as each process operates within its own protected memory space. This isolation improves robustness and fault containment, since errors in one process cannot directly corrupt the state of another. However, inter-process communication generally incurs higher overhead, as it requires explicit mechanisms to exchange data across process boundaries [22].

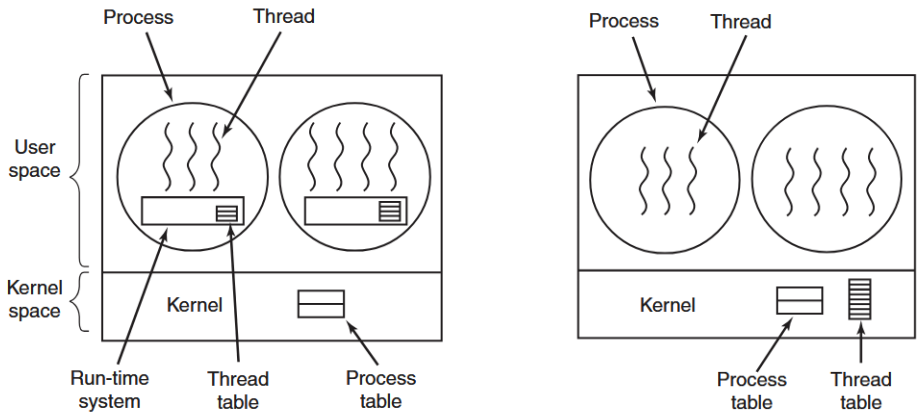
From an execution perspective, processes constitute units of concurrency that provide strong isolation, at the cost of increased management and communication overhead.

2.4.2.2 Threads

A thread represents a lower-level unit of execution within a process. Multiple threads within the same process share the process address space and resources, while maintaining independent execution contexts such as program counters and stacks [23]. This sharing enables efficient communication and cooperation between concurrent activities, as data can be accessed directly without inter-process communication mechanisms.

Threads can be classified according to how their management is implemented. In user-level threading models, thread creation, scheduling, and synchronization are handled by runtime systems operating entirely in user space. As depicted in Fig. 2.5(a), the kernel is unaware of individual threads and schedules only processes, while thread tables and scheduling decisions are maintained by the runtime environment. This approach enables fast thread management but limits the ability to exploit multiple processors transparently [22].

In contrast, kernel-level threads, shown in Fig. 2.5(b), are managed directly by the operating system kernel. In this model, the kernel maintains thread tables and schedules threads independently, allowing threads belonging to the same process to be distributed across multiple processing units. While this enables true parallel execution on multi-core systems, it also introduces higher management overhead due to kernel involvement [21].



(a) User-level threading model

(b) Kernel-level threading model

Figure 2.5: Conceptual comparison between user-level and kernel-level thread management. In user-level threading, thread scheduling and management are handled by a runtime system in user space, whereas in kernel-level threading these responsibilities are managed directly by the operating system kernel. Adapted from [23].

In both models, threads within a process typically share memory and other resources, adopting a shared-memory execution model. While this model supports efficient cooperation, it also requires

careful coordination to ensure correct access to shared data. The mechanisms used to address these challenges are discussed in the following subsection.

2.4.3 Synchronization and Shared Resource Access

In concurrent execution models, multiple threads or tasks may access shared resources during their execution. While shared access enables cooperation and efficient communication, it also introduces challenges related to correctness, as uncontrolled interactions may lead to inconsistent or unintended system states.

2.4.3.1 Race Conditions

In concurrent execution models, incorrect synchronization may result in race conditions, where the behavior of a computation depends on the timing of concurrent executions. When multiple threads access shared resources without proper coordination, the final system state may vary across different execution schedules, even when the same operations are performed [21].

Race conditions are particularly difficult to detect and reason about, as they often arise only under specific execution orders of concurrent tasks.

2.4.3.2 Critical Sections

To prevent race conditions, concurrent systems identify regions of execution that access shared resources as critical sections. A critical section denotes a portion of code in which a shared resource is accessed or modified and whose execution must be mutually exclusive.

Correct concurrent execution requires that critical sections operating on the same shared resource are not executed simultaneously by multiple threads, thereby ensuring consistency of the shared state [23].

2.4.3.3 Mutexes

Mutual exclusion mechanisms, commonly referred to as mutexes, provide a concrete means to enforce exclusive access to critical sections. A mutex ensures that at most one thread may enter a given critical section at any time.

By serializing access to shared resources, mutexes prevent concurrent modifications that could otherwise result in inconsistent system states. This behavior is illustrated in Fig. 2.6, where one process is blocked while another executes a critical section.

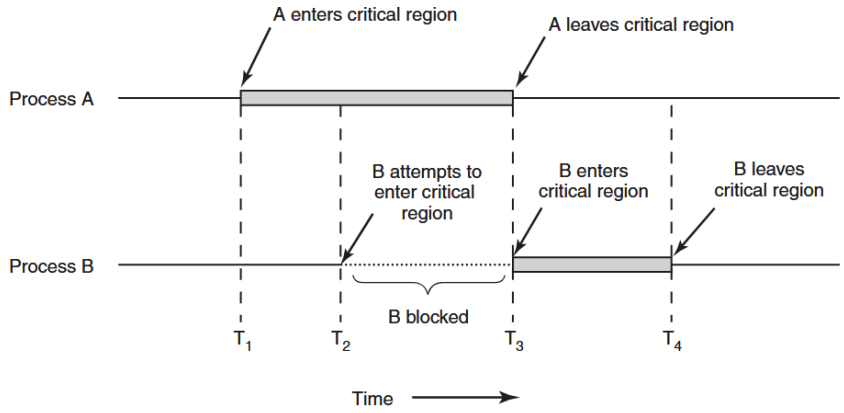


Figure 2.6: Illustration of mutual exclusion in a critical section. While Process A executes its critical section, Process B is blocked until the resource becomes available [23].

2.4.3.4 Semaphores

Semaphores provide a general synchronization for controlling access to shared resources and enforcing ordering constraints between concurrent tasks. Unlike mutexes, which are primarily intended to enforce exclusive access, semaphores support more flexible coordination patterns [24].

According to Arpaci-Dusseau [21], semaphores are commonly classified into two types:

- **Binary semaphores**, whose value is restricted to zero or one, and which can be used to enforce mutual exclusion in a manner similar to mutexes, though without ownership constraints;
- **Counting semaphores**, which take non-negative integer values and are used to manage access to a finite number of identical resources. The semaphore value represents the number of currently available resource instances and is decremented each time a resource is allocated to a process or thread.

2.4.3.5 Deadlocks

Another fundamental challenge in concurrent systems is deadlock, a state in which a set of threads becomes permanently blocked because each thread is waiting for resources held by others. Deadlocks arise from circular waiting dependencies and result in a loss of system progress, requiring careful design and coordination to avoid [22].

2.5 Summary

This chapter introduced the formal foundations of Multi-Agent Path Finding, including problem definition, environment representation, and classical graph search methods. In addition, fundamental concepts from parallel and concurrent computing were presented to motivate scalable execution

and coordination strategies. These elements together provide the theoretical basis for the MAPF algorithms analyzed and proposed in the remainder of this dissertation.

Chapter 3

State of the Art

This chapter reviews the state of the art in MAPF, focusing on the main coordination paradigms and algorithmic approaches proposed in the literature. Section 3.1 surveys decentralized, centralized, hybrid, and learning-based coordination strategies, highlighting their respective strengths and limitations in multi-robot systems. Section 3.2 examines agent prioritization heuristics for sequential MAPF planners, with particular attention to their impact on solution feasibility and performance. Finally, Section 3.3 presents the CRIIS fleet coordination system, which operationalizes centralized MAPF planning and serves as the base for the methods investigated in this work.

3.1 Algorithmic Approaches to MAPF

The MAPF problem has been addressed through different coordination paradigms, reflecting distinct assumptions about system architecture and the distribution of coordination responsibilities. In multi-robot systems and fleet coordination, these paradigms exhibit characteristic trade-offs in terms of coordination quality, computational complexity, and robustness.

This chapter considers the following coordination paradigms:

- **Decentralized coordination**;
- **Centralized coordination**;
- **Hybrid coordination**;
- **Learning-based coordination**.

The following subsections discuss each of these paradigms in turn, introducing their main coordination principles and presenting representative algorithmic approaches to illustrate how these paradigms are realized in practice, with particular emphasis on centralized approaches, which constitute the algorithmic focus of this document.

3.1.1 Decentralized Coordination

Decentralized approaches distribute planning and decision-making among the agents themselves, without relying on a global coordination authority. In this paradigm, agents typically operate based on local information and, when available, limited communication with neighboring agents. Coordination emerges through local interactions, reactive behaviors, or distributed decision-making mechanisms, as illustrated in Figure 3.1.

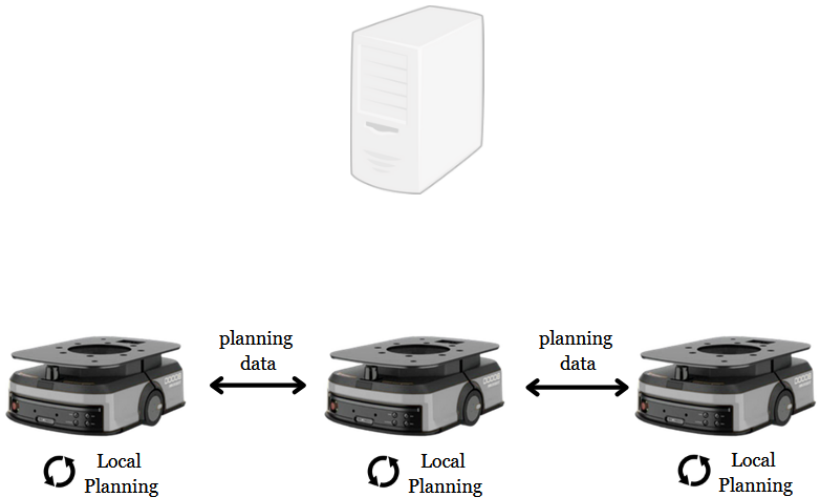


Figure 3.1: Illustration of a decentralized coordination architecture, where agents communicate directly with each other and no central planning entity is responsible for computing globally consistent paths.

Such approaches are particularly attractive in large-scale systems or in scenarios where communication is unreliable, as they avoid centralized computational bottlenecks and single points of failure [25]. Decentralized coordination has been extensively studied in the context of autonomous mobile robots, where agents must operate under partial observability and limited global knowledge [26].

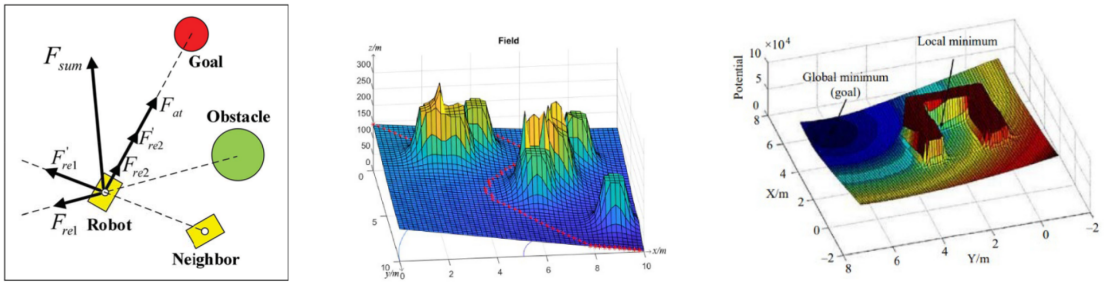
However, the absence of a global planning entity makes it difficult to ensure globally consistent behavior. Decentralized MAPF approaches generally provide limited guarantees with respect to completeness, deadlock avoidance, or solution quality, especially in environments with narrow passages or strong agent interactions [27]. As a result, while decentralized paradigms offer scalability, they often struggle to meet the predictability and safety requirements of tightly coordinated robotic fleets.

Several algorithmic approaches exemplify decentralized coordination in multi-robot systems. Reactive collision avoidance methods rely exclusively on local sensing and agent–agent interactions to prevent collisions, enabling scalable coordination without global planning, although with limited guarantees [28]. Distributed planning and negotiation-based approaches further extend this paradigm through peer-to-peer information exchange and local conflict resolution, improving robustness while avoiding centralized control [27].

A representative example of decentralized coordination is provided by Artificial Potential Field (APF) methods. Originally proposed for single-agent navigation, APF approaches were designed to guide an agent towards a goal while avoiding obstacles by defining attractive and repulsive potential functions over the workspace [29]. More recent extensions have adapted APF-based methods to multi-robot settings by incorporating interaction forces between agents, which are treated as dynamic obstacles or sources of repulsive potentials. In such approaches, coordination emerges from local interactions, as each agent computes its motion based solely on locally available information. An example of this paradigm is the APF-CPP approach proposed by Wang et al. [30], where artificial potential fields are used for online multi-robot coverage path planning in a fully decentralized manner.

The resulting motion emerges from the combination of attractive forces towards goals and repulsive forces arising from obstacles and neighboring agents, allowing agents to react continuously to their local environment without requiring global coordination, as illustrated in Figure 3.2a.

Although Figures 3.2b and 3.2c do not explicitly represent a multi-agent setting, they are highly illustrative of the fundamental behavior of potential fields and the emergence of local minima. Figure 3.2b depicts an idealized potential field, however, the interaction between attractive and repulsive potentials often gives rise to undesired local minima, as shown in Figure 3.2c. These local minima constitute a well-known limitation of APF-based methods, as agents may become trapped despite the existence of a feasible path to the goal [31].



(a) Attractive and repulsive forces acting on an agent in a multi-robot setting [30].

(b) Potential field [31].

(c) Potential field exhibiting local minima induced by obstacles [31].

Figure 3.2: Illustration of APF concepts.

Overall, the APF-based approach discussed above illustrates well several characteristic limitations of decentralized coordination methods. In particular, the reliance on local and reactive decision-making can lead to undesired behaviors such as local minima and deadlocks, reflecting the difficulty of achieving globally consistent solutions. As a result decentralized approaches typically provide limited guarantees regarding global completeness or solution quality.

3.1.2 Centralized Coordination

Centralized MAPF approaches rely on a single planning entity with access to the complete global state of the system, including the environment representation and the start and goal locations of all agents. This entity is responsible for computing coordinated, collision-free trajectories that ensure globally consistent behavior across the fleet. A typical centralized coordination architecture is depicted in Figure 3.3.

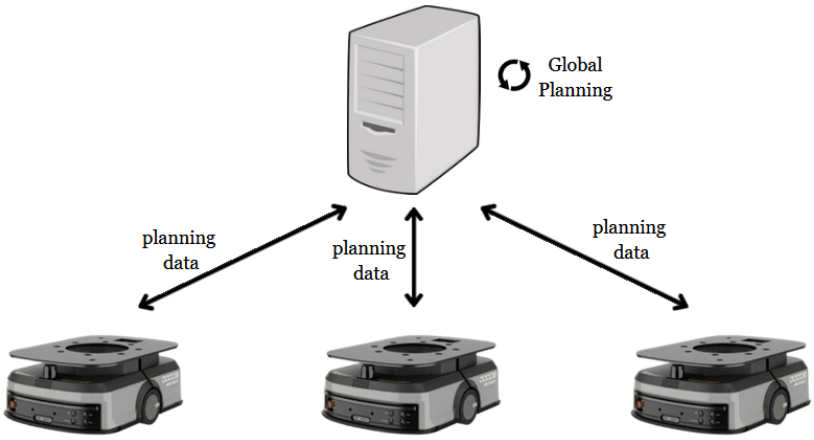


Figure 3.3: Illustration of a centralized coordination architecture, in which a central planning entity computes coordinated, collision-free trajectories for all agents based on complete global information.

Centralized coordination has long been adopted in industrial and structured environments, where reliable communication and centralized supervision are available [32]. This paradigm enables strong guarantees regarding safety, predictability, and coordination quality, and aligns well with fleet management systems that incorporate execution monitoring and replanning capabilities.

From a computational perspective, centralized MAPF approaches face inherent scalability challenges as the number of agents increases. As discussed in the background, the MAPF problem is NP-hard under standard formulations, which implies that planning rapidly becomes computationally expensive in densely populated or highly constrained environments. Consequently, practical centralized planners must carefully balance solution quality with computational efficiency. Nevertheless, access to complete global information allows centralized methods to reason explicitly about agent interactions and to resolve conflicts in a systematic and predictable manner, which remains a key advantage in scenarios requiring strong coordination guarantees.

Several algorithmic approaches have been proposed for centralized MAPF planning, differing in how conflicts are represented, detected, and resolved. Throughout this subsection, the discussion focuses on two representative methods, TEA* [33] and Conflict-Based Search (CBS) [3], which exemplify distinct strategies for centralized coordination.

3.1.2.1 TEA*

TEA* is a centralized multi-robot coordination approach that builds directly upon the classical A* search algorithm described in the background by extending the search space with an explicit temporal dimension. Instead of reasoning solely over spatial states, TEA* models robot motion in a space–time representation, where each state is augmented with a discrete time index, enabling the planner to reason explicitly about future agent positions and to avoid conflicts [33].

In this formulation, time progresses in discrete steps that correspond to the traversal of graph edges. Each feasible motion between two adjacent spatial nodes induces a transition to the next temporal layer, and the search is propagated forward up to a predefined temporal horizon $k_{\max}$. This layered space–time graph, illustrated in Figure 3.4, allows TEA* to encode both spatial feasibility and temporal ordering directly within the search process. By reserving nodes and edges across successive time layers during planning, the algorithm ensures that agents do not occupy the same resource at the same time, preventing future conflicts a priori rather than detecting and resolving them after paths have been computed.

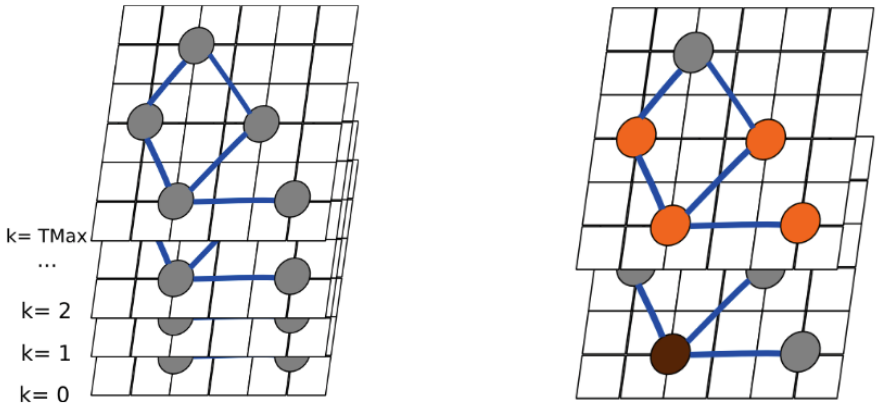


Figure 3.4: Illustration of the TEA* space–time representation [34].

In TEA*, paths are planned sequentially for each agent over a shared environment representation, which is commonly modeled as a graph. Once a path is computed for a given agent, the corresponding space–time trajectory is reserved and effectively treated as a dynamic obstacle for subsequently planned agents. During planning, nodes and edges occupied at specific time steps are marked as unavailable, preventing later agents from generating paths that would conflict with already planned motions. This temporal reservation mechanism allows TEA* to generate collision-free solutions, without requiring an explicit conflict detection and resolution stage [34].

A key characteristic of TEA* is its suitability for online execution and replanning. By maintaining a time-indexed representation of resource usage, the planner can update or recompute paths in response to execution delays, dynamic disturbances, or unexpected agent behavior. This capability makes TEA* particularly attractive for industrial and warehouse environments, where strict coordination requirements coexist with execution uncertainty and frequent replanning needs [33].

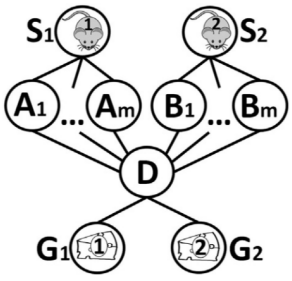
Several extensions of the original TEA* formulation have been proposed to address limitations arising from simplifying assumptions in the base model. In particular, Moura et al. [6] introduce a temporal optimization that accounts for robot kinematics, such as orientation changes and rotation costs, reducing the mismatch between the discrete planning model and real-world execution for differential-drive robots. Other works address livelock and deadlock phenomena by dynamically adjusting planning priorities or reordering the sequence in which agents are replanned [34].

Overall, the sequential nature of TEA* implies that the resulting solution may depend on the order in which agents are planned, potentially affecting solution quality or feasibility in densely constrained environments. This limitation, however, can be mitigated through the adoption of appropriate planning heuristics and agent ordering strategies. While this design choice sacrifices optimality guarantees under standard MAPF cost formulations, it offers a favorable trade-off in terms of computational efficiency.

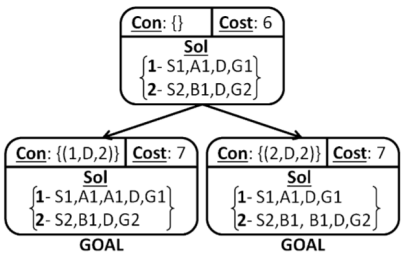
3.1.2.2 CBS

CBS is a centralized MAPF algorithm that resolves coordination through a hierarchical planning framework composed of a high-level conflict resolution process and a low-level path planner [3]. The key idea underlying CBS is to decouple individual path planning from conflict management, allowing agents to plan largely independently while explicitly resolving interactions when conflicts arise.

Figure 3.5a illustrates a simple MAPF instance in which two agents navigate a shared environment with alternative routes between start and goal locations. Although individual shortest paths may exist for each agent, conflicts can arise when agents attempt to occupy the same location or traverse the same edge at the same time, motivating the need for explicit coordination.



(a) Example MAPF instance illustrating potential conflicts between agents navigating a shared environment.



(b) Conflict Tree expansion in CBS, where conflicts are resolved by branching over alternative constraints.

Figure 3.5: Overview of CBS. (a) A MAPF instance in which independently planned paths may lead to conflicts. (b) High-level Conflict Tree used by CBS to systematically resolve conflicts through constraint generation. Adapted from [3].

The high-level search maintains a Conflict Tree (CT), illustrated in Figure 3.5b, where each node corresponds to a set of constraints and associated individual paths. When a conflict is detected, the corresponding CT node is expanded by generating alternative constraint sets, each prohibiting

one of the conflicting agents from performing the conflicting action. While this mechanism guarantees completeness and optimality under standard cost formulations, the number of conflicts and CT nodes may grow exponentially in the worst case, motivating several extensions to improve scalability.

At the low level, each agent computes a shortest path subject to a set of constraints. Recent extensions of CBS adopt Safe Interval Path Planning (SIPP) as the low-level planner in order to handle continuous time. This combination, commonly referred to as Continuous-Time CBS (CCBS), integrates SIPP into the CBS framework to enable optimal planning without discretizing time [35], [36]. SIPP operates by representing, for each spatial location, intervals of time during which the location is safe to occupy, allowing waiting actions and motion durations to be handled efficiently under temporal constraints. This integration makes CBS particularly well suited for domains where temporal feasibility plays a central role [36].

Several other variants of CBS have been proposed to address these limitations:

- **ECBS:** Bounded-suboptimal CBS introduces focal search to trade optimality for efficiency, allowing solutions within a predefined suboptimality bound [37].
- **CBSH:** Heuristic-guided CBS augments the high-level search with admissible heuristics that estimate the cost of unresolved conflicts, significantly reducing the size of the conflict tree while preserving optimality [38].
- **CBSH-RTC:** CBS variant with symmetry reasoning mitigate the combinatorial explosion of collision resolutions by identifying and pruning pairwise symmetric path combinations, substantially improving scalability and search efficiency. [39].

Overall, CBS and its variants exemplify a conflict-driven coordination paradigm, in which interactions between agents are explicitly detected and resolved through constraint generation at the high level.

3.1.2.3 Contrasting Centralized Coordination Strategies

Although both TEA* and CBS are centralized approaches to multi-agent path finding, they embody fundamentally different coordination strategies. CBS follows a conflict-resolution paradigm, in which agents initially plan independently and conflicts are detected and resolved through constraint generation at a higher level. As a result, coordination emerges from the iterative refinement of individual plans in response to detected interactions.

In contrast, TEA* adopts a conflict-avoidance strategy by reserving spatial resources across discrete time layers, TEA* prevents conflicts by construction, rather than detecting and resolving them after paths have been generated. These contrasting design choices lead to different trade-offs in terms of computational complexity, solution optimality, and suitability for online execution, motivating their joint analysis in applied fleet management scenarios.

3.1.3 Hybrid Coordination

Hybrid MAPF approaches seek to combine elements of decentralized and centralized coordination in order to leverage the advantages of both paradigms. Typically, these approaches adopt a hierarchical structure, in which high-level coordination decisions are made centrally, while lower-level planning or execution decisions are delegated to individual agents, as illustrated in Figure 3.6.

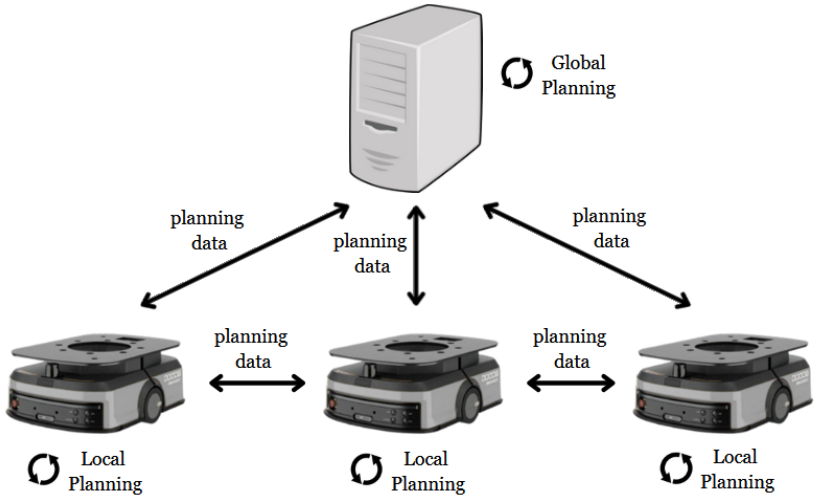


Figure 3.6: Illustration of a hybrid coordination architecture combining centralized decision-making at higher levels with decentralized planning or execution at the agent level.

The motivation for hybrid coordination lies primarily in scalability. By abstracting parts of the problem or limiting centralized planning to higher-level decisions, hybrid approaches aim to reduce the computational burden associated with fully centralized coordination, while still preserving some degree of global consistency [40].

Despite their potential benefits, hybrid and hierarchical approaches introduce additional complexity and may compromise theoretical guarantees. In particular, the separation between planning layers can lead to suboptimal solutions or coordination failures in complex or highly constrained environments. As such, hybrid paradigms are often tailored to specific application domains and system architectures.

Several representative approaches illustrate this hybrid coordination paradigm. Hierarchical MAPF methods employ centralized planning at a coarse level, such as region or waypoint assignment, while delegating fine-grained path planning to individual agents [41]. Other hybrid strategies rely on centralized global guidance combined with decentralized local execution, allowing agents to react autonomously to local disturbances while respecting global coordination constraints [40]. Region-based coordination approaches further reduce complexity by enforcing centralized constraints at the level of areas or traffic corridors, while allowing agents to plan locally within these regions [42], [43].

3.1.4 Learning-Based Coordination

More recently, learning-based approaches have been explored as an alternative paradigm for multi-agent coordination and path planning. These methods typically rely on machine learning techniques, such as reinforcement learning or imitation learning, to learn coordination policies from data rather than explicitly computing plans through search or optimization.

Learning-based coordination is motivated by the potential to generalize across environments and to adapt to complex, dynamic scenarios. In multi-agent settings, learning-based methods have been applied to collision avoidance, cooperative navigation, and decentralized decision-making [44], [45].

Representative examples of learning-based coordination include decentralized multi-agent reinforcement learning approaches, where agents learn local navigation and collision avoidance policies through interaction with the environment and other agents, without relying on explicit global planning [46]. Such methods emphasize scalability and adaptability, but typically lack formal guarantees.

Another prominent line of work combines classical MAPF planners with learning-based components. In these hybrid approaches, learning is used to guide heuristic selection, conflict resolution, or priority assignment within search-based planners, improving efficiency while preserving the underlying planning structure [44].

Despite promising results, learning-based approaches face significant challenges in the context of MAPF. These include the need for large amounts of training data and the difficulty of providing strong guarantees regarding safety, completeness, or optimality. As a result, learning-based coordination is often considered complementary to classical MAPF methods rather than a direct replacement, particularly in safety-critical fleet management applications.

3.2 Agent Prioritization Heuristics

Sequential and prioritized planning approaches constitute an important class of MAPF solvers, particularly in centralized settings where agents are planned one after another according to a predefined priority ordering. As discussed in the previous section, TEA* follows this paradigm by planning agents sequentially in a shared space–time representation. A well-known characteristic of such approaches is their strong dependence on the chosen agent ordering, which can significantly influence solution feasibility, solution quality, and computational performance. This sensitivity has motivated extensive research on agent prioritization heuristics, aiming to determine planning orders that reduce conflicts, deadlocks, or excessive delays. However, as consistently highlighted in the literature and recent surveys, no single heuristic dominates across all environments and task configurations, with performance being highly scenario-dependent [5], [47]. This section reviews representative classes of agent prioritization heuristics proposed in the context of prioritized MAPF planning.

3.2.1 Randomized and Multi-Ordering Strategies

The simplest approach to mitigate the sensitivity of sequential MAPF planners to agent ordering is randomized prioritization, which is commonly used as a baseline strategy. In this approach, agents are assigned priorities according to random permutations, and planning is repeated multiple times. Despite its simplicity, randomized ordering can occasionally outperform carefully designed heuristics by avoiding systematic bias toward unfavorable priority configurations [47]. As such, it is frequently adopted as a robustness baseline in empirical evaluations.

3.2.2 Path-Length-Based Heuristics

One of the most widely used classes of prioritization heuristics is based on estimated path length. The underlying intuition is that agents with longer or more constrained paths should be planned earlier, as they typically have fewer alternative routing options.

A common variant is the *Longest Path First* heuristic, where agents are ordered in descending order of their shortest-path distance to the goal [5]. Conversely, *Shortest Path First* heuristics prioritize agents with shorter paths, favoring early completion but often increasing the likelihood of conflicts for agents with longer routes. While path-length-based heuristics are simple to compute and effective in sparse environments, they do not explicitly account for interactions between agents and may perform poorly in congested or highly coupled scenarios.

3.2.3 Conflict-Aware Prioritization

To address the limitations of purely distance-based heuristics, several approaches incorporate conflict awareness into the prioritization process. These heuristics estimate the likelihood of an agent being involved in conflicts based on spatial overlap with other agents' paths or shared critical regions.

Early approaches prioritize agents whose shortest paths intersect frequently with others, under the assumption that planning them first reduces downstream conflicts [47]. While conflict-aware heuristics often improve success rates in dense environments, they introduce additional computational overhead and typically rely on approximate conflict estimation.

3.2.4 Environment-Aware and Bottleneck-Based Heuristics

Another class of prioritization heuristics exploits structural properties of the environment. Agents whose paths traverse narrow corridors, bottlenecks, or shared resources are prioritized earlier, as these regions are particularly prone to congestion and deadlocks.

Such heuristics are especially relevant in structured environments such as warehouses or factories, where spatial constraints strongly influence coordination difficulty [47]. Environment-aware prioritization has been shown to reduce deadlocks and improve execution predictability, particularly when combined with time-augmented planning approaches.

3.3 Fleet Coordination System

This section describes the architecture of the CRIIS fleet coordination system, which is designed as a centralized coordination framework built around TEA*, complemented with supervision and graph-processing mechanisms to handle real-world scenarios [4]. The system is implemented on top of the Robot Operating System (ROS), which provides the communication middleware and execution infrastructure supporting both centralized coordination and distributed robot control.

The architecture follows a modular design that separates global coordination responsibilities from local robot control. As illustrated in Figure 3.7, at a high level, the system is composed of two main components. The *Central Coordination Module* is responsible for global planning, supervision, and coordination, maintaining a global view of the environment and the fleet state. In contrast, the *Agent Control Module*, instantiated on each robot, is responsible for executing the received plans, interfacing with the robot's navigation stack, and providing execution feedback to the central supervisor [4].

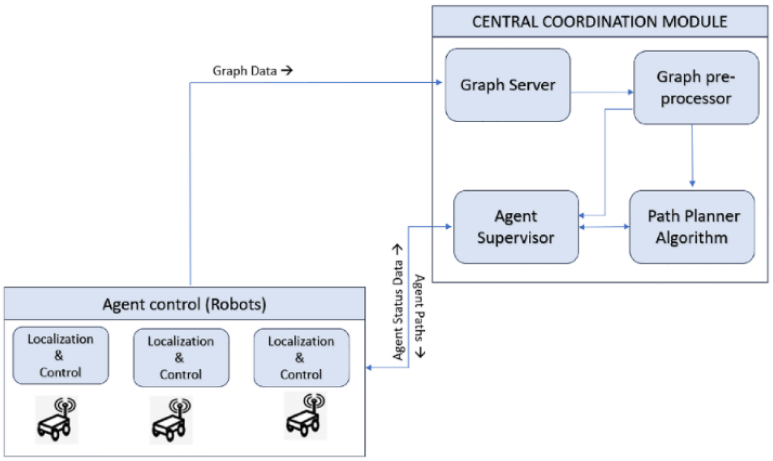


Figure 3.7: High-level architecture of the CRIIS fleet coordination system [4].

The remainder of this section focuses on a detailed description of the Central Coordination Module and its internal components.

3.3.1 Central Coordination Module

The Central Coordination Module is responsible for global decision-making and coordination across the robotic fleet. It maintains a centralized representation of the environment and agent states, computes coordinated plans, and supervises execution[4].

As summarized in Figure 3.8, this module comprises four core components: the Graph Server, the Graph Pre-Processor, the Path Planner, and the Supervisor. Interaction with the robots' navigation systems is mediated through a Navigation Handler, which provides an abstraction layer for plan dispatch and execution feedback.

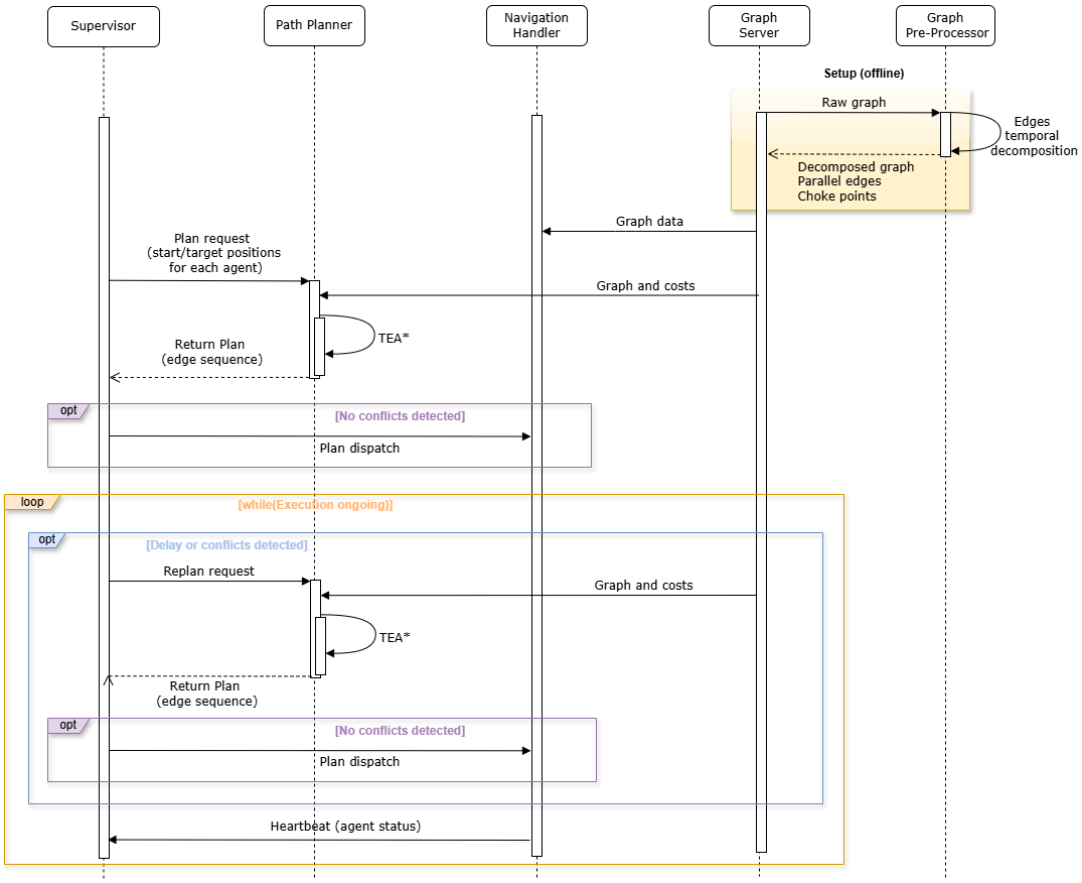


Figure 3.8: Sequence diagram of the CRIIS fleet coordination workflow.

3.3.1.1 Graph Server

The Graph Server is responsible for storing and providing access to the global navigation graph shared by all agents in the fleet. This graph encodes the topological structure of the environment, including nodes, directed edges, and their associated traversal costs. In the CRIIS fleet coordination system, these costs are expressed in temporal terms, enabling time-aware planning and synchronization among agents [4].

Additionally, the Graph Server exposes a query interface that allows other components, namely the Path Planner, to retrieve graph topology, edge attributes, and metadata required for coordination. By decoupling graph management from planning logic, the system promotes modularity and facilitates future extensions, such as dynamic cost updates or graph augmentation strategies.

3.3.1.2 Graph Pre-Processor

The Graph Pre-Processor operates during an offline configuration phase and augments the raw navigation graph with information required for multi-robot coordination, transforming a purely geometric representation into a coordination-aware graph [4]. Its responsibilities include identifying footprint-related constraints, such as parallel edges that cannot be safely occupied simultaneously,

and detecting critical shared resources (choke points) like narrow corridors or doorways that are prone to congestion and deadlocks.

Additionally, since TEA* advances time in discrete steps associated with edge traversals, the pre-processor performs edge discretization and normalization according to a global temporal resolution. This ensures consistent traversal times across the graph, improving temporal alignment between agents and enabling more accurate space–time reasoning during planning, thereby reducing synchronization errors and unnecessary replanning [4].

3.3.1.3 Path Planner

The Path Planner is the core decision-making component responsible for computing coordinated, collision-free trajectories for the entire fleet. In the CRIIS system, this functionality is implemented using the TEA* algorithm [4].

Planning requests include the current symbolic positions and target goals of all agents. Upon receiving a request, the Path Planner retrieves the navigation graph and associated traversal costs from the Graph Server and computes time-consistent sequences of edges for each agent. By reasoning over a shared space–time representation, the planner can explicitly account for interactions between agents, reserving graph resources over time and preventing conflicts through temporal exclusion mechanisms.

The centralized nature of the Path Planner enables globally coherent solutions, particularly in environments with shared resources or high robot density. This approach allows conflicts to be resolved at planning time rather than during execution, reducing the likelihood of deadlocks, emergency stops, or excessive local replanning [4].

3.3.1.4 Navigation Handler

The Navigation Handler serves as the integration layer between the centralized coordination logic and the robots' low-level navigation systems. It exposes an API that supports navigation requests, plan dispatch, translation of symbolic graph-based edge sequences into commands compatible with the robot navigation stack, and continuous monitoring of execution progress.

In particular, the Navigation Handler provides a planning service through which a robot can be requested to navigate to a given target vertex. When this service is called, the Navigation Handler forwards the request to the Supervisor, which in turn invokes the Path Planner to compute a feasible path within the shared graph representation. Once the resulting plan is received, the Navigation Handler translates the symbolic sequence of graph edges into navigation commands executable by the robot.

During execution, the Navigation Handler collects feedback such as the current symbolic state, edge completion status, and execution delays, and reports this information to the Supervisor. By isolating execution-specific concerns from planning logic, it enhances system modularity and allows the coordination framework to remain agnostic to the underlying robot platforms and navigation implementations [4].

3.3.1.5 Supervisor

The Supervisor is responsible for orchestrating the overall coordination loop by closing the feedback cycle between planning and execution. It aggregates execution feedback provided by the Navigation Handler, monitors the progress of each agent, and maintains a global view of the system state [4].

Based on this information, the Supervisor detects abnormal or unexpected conditions, such as execution delays, deviations between planned and executed trajectories, or emerging conflicts caused by dynamic disturbances. When such situations are identified, the Supervisor triggers replanning requests to the Path Planner.

This supervision mechanism is a key contributor to system robustness under real-world uncertainty, including variable execution times, communication latency, and dynamic obstacles. By enabling adaptive replanning while preserving centralized coordination, the Supervisor ensures that the fleet can maintain safe and efficient operation even when assumptions made at planning time are violated [4].

3.4 Summary

This chapter reviewed the main coordination paradigms and algorithmic approaches to MAPF, with particular emphasis on centralized planning methods. While decentralized and learning-based approaches offer scalability and adaptability, their limited guarantees regarding safety, predictability, and solution quality restrict their applicability in tightly coordinated fleet management scenarios.

Motivated by these constraints, the CRIIS fleet coordination system adopts a centralized, time-aware coordination architecture that explicitly reasons about agent interactions and execution timing. By integrating TEA* within a broader framework that includes graph preprocessing, supervision, and execution monitoring, CRIIS bridges the gap between theoretical MAPF formulations and the practical requirements of real-world robotic fleets, such as execution uncertainty, communication delays, and physical interaction constraints.

A central design choice in CRIIS is the use of sequential, prioritized planning, which enables responsive and scalable coordination but introduces a strong dependence on agent ordering. Although numerous prioritization heuristics have been proposed in the literature, recent surveys indicate that agent prioritization remains comparatively underexplored despite its practical relevance. This observation directly motivates the investigation of alternative prioritization strategies and multi-ordering approaches pursued in this work.

Chapter 4

Baseline System Stabilization

Before introducing new prioritization heuristics and parallel planning strategies, it was first necessary to stabilize the baseline coordination pipeline. Preliminary real-time experiments revealed temporal inconsistencies in the interaction between the robots' Navigation Handlers, the Supervisor, and the Path Planner during planning and replanning operations.

This chapter analyzes the root causes of these inconsistencies and describes the modifications introduced to mitigate them, starting with the instability issues observed in the baseline system in Section 4.1, followed by a temporal analysis of the coordination pipeline in Section 4.2, and concluding with a summary of the stabilization mechanisms and their role in providing a reliable baseline for the following chapters in Section 4.3.

4.1 Initial System Instability Issues

During planning and replanning operations, robots occasionally received paths whose initial segments no longer matched their current physical positions. This mismatch caused the navigation controller to observe a large error between the robot's current pose and the first target segment of the newly received path. When this error exceeded the controller's admissible threshold, the robot could stop instead of continuing the planned motion, potentially compromising the consistency and safety of the overall coordination process.

A preliminary analysis revealed that these inconsistencies were strongly related to the way planning requests and robot state updates were processed by the Supervisor. More specifically, these operations could be triggered using outdated or temporally inconsistent robot states. This issue became particularly relevant because the planner may require a non-negligible amount of computation time, especially in large environments or high density multi-robot scenarios. Consequently, the discrepancy between the planner's internal representation of the system and the real robot states could further increase before the generated plan was finally applied.

These observations motivated a deeper analysis of the temporal behavior of the coordination pipeline, with particular focus on message propagation delays, ROS callback queue processing, and replanning triggering mechanisms.

4.2 Temporal Analysis of the Coordination System

To better understand the origin of the inconsistencies described above, a temporal analysis of the coordination pipeline was conducted. Figure 4.1 presents a simplified representation of the most relevant events and temporal intervals involved in the interaction between the robots and the Coordination System.

The abstraction focuses on the temporal intervals associated with message transmission, ROS callback queue processing, and planner execution. In particular:

- $\Delta t_{1,i}$ corresponds to the transmission delay between robot i and the Coordination System. The transmitted messages may correspond either to planning requests or to robot state updates. Both message types are relevant in this context, since planning requests directly trigger planning operations, while robot state updates may trigger replanning when inconsistencies in the robots' execution progress are detected;
- $\Delta t_{2,i}$ represents the waiting time of incoming messages in the ROS callback queue before being processed by the Supervisor;
- $\Delta t_{3,i}$ denotes the planner execution time required to compute a new plan after a planning or replanning request is processed;
- $\Delta t_{4,i}$ corresponds to the transmission delay associated with sending newly computed plans from the Supervisor back to robot i.

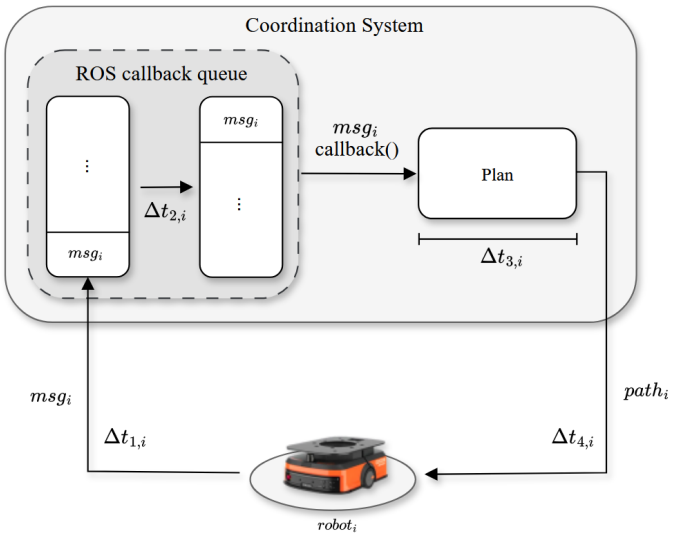


Figure 4.1: Simplified representation of the temporal flow associated with message processing and planning operations within the coordination system.

Since the experiments were conducted in a fully local simulation environment, with all ROS nodes executing on the same machine, inter-process communication delays were considered negligible with respect to planner execution time. Therefore, the dominant sources of temporal

inconsistency were mainly associated with sequential ROS callback processing and planner execution, corresponding primarily to the intervals $\Delta t_{2,i}$ and $\Delta t_{3,i}$. The following subsections focus on these two intervals and describe the main limitations identified in the baseline Coordination System.

4.2.1 Redundant Planner Executions

One of the main issues identified in the baseline coordination system was the redundant execution of the planner when multiple robots issued planning requests within a short time interval. In the coordination framework, each robot can individually request a plan to reach a given target vertex. However, in the original implementation, each planning request received by the Supervisor was immediately forwarded to the Path Planner, independently of whether requests from other robots had arrived at approximately the same time.

When a planning request was processed, the Supervisor requested the Path Planner to compute a coordinated plan for the set of robots requiring coordination at that instant. This set included the robots that had already submitted planning requests and had not yet reached their assigned destinations. Consequently, several planner executions could be triggered sequentially for what was effectively a single planning event, with the first planning execution considering only the first processed robot request, the second considering the first two requests, and this pattern continuing until the final execution.

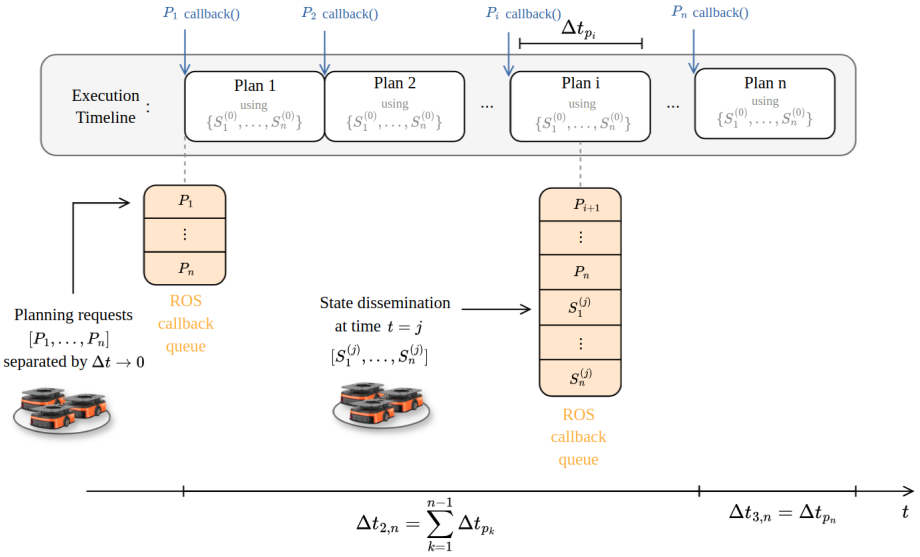


Figure 4.2: Sequential processing of nearly simultaneous planning requests, resulting in redundant planner executions.

As illustrated in Figure 4.2, the planning requests $\{P_1,\ldots,P_n\}$ are sent by the robots within a very short time interval. These requests are inserted sequentially into the ROS callback queue and processed one at a time by the Supervisor. In the baseline implementation, each processed request immediately triggers a new planner execution, leading to a sequence of n planning processes.

During this sequence, Plan 1 is computed after processing only P_1 and, more generally, Plan i after processing $\{P_1, \ldots, P_i\}$. However, all these plans are computed using the same initial robot state set $\{S_1^{(0)}, \ldots, S_n^{(0)}\}$, corresponding to the system configuration available before the first planning request was processed. Only Plan n considers the complete request set $\{P_1, \ldots, P_n\}$.

This behavior affected both computational efficiency and temporal consistency. The first $n-1$ planner executions provided limited practical value, since each intermediate plan was immediately superseded by a later execution considering a larger and more complete set of robot requests. At the same time, these redundant executions blocked the ROS callback queue, delaying the processing of newly received robot state updates. As also shown in Figure 4.2, a new state dissemination may occur at time $t = j$ during the execution of planning process i, producing updated robot states $\{S_1^{(j)}, \ldots, S_n^{(j)}\}$. However, if the corresponding state update messages are inserted after pending planning requests in the ROS callback queue, they remain unprocessed until the previous planner executions have finished.

This is particularly problematic because the robots associated with the first $i-1$ planning executions may already have received and started executing their newly computed paths. Consequently, their actual states at time $t = j$ may differ from the states $S^{(0)}$ used when those paths were computed.

In this scenario, the delay affecting the state information used by the final planning process can be approximated by the accumulated execution time of the previous planner calls:

$$\Delta t_{2,n} = \sum_{k=1}^{n-1} \Delta t_{p_k} \tag{4.1}$$

, where Δt_{p_k} denotes the execution time of the k-th planning process. This accumulated waiting time is then followed by the computation time of the final planning process itself:

$$\Delta t_{3,n} = \Delta t_{p_n} \tag{4.2}$$

Assuming that the message transmission delays $\Delta t_{1,n}$ and $\Delta t_{4,n}$ are negligible in the local simulation environment, the total response time between the moment the first planning request is issued and the moment the final corresponding plan is delivered can be approximated as:

$$\Delta t_{\text{response},n} \approx \Delta t_{2,n} + \Delta t_{3,n} \tag{4.3}$$

For the final planning process, this results in:

$$\Delta t_{\text{response},n} \approx \sum_{k=1}^{n-1} \Delta t_{p_k} + \Delta t_{p_n} = \sum_{k=1}^{n} \Delta t_{p_k}. \tag{4.4}$$

As a consequence, the time required to generate a plan that considers all robots may increase significantly. Moreover, the resulting plan may be computed using state information that no longer accurately reflects the actual positions of the robots in the system.

4.2.2 Planning with Temporally Inconsistent Robot States

A second issue identified in the baseline Coordination System was related to the way robot state updates were processed by the Supervisor. In the original implementation, whenever a robot sent a state update, the internal variables used to track that robot's progress were immediately updated. The updated progress was then compared with the progress of the remaining robots to determine whether a replanning operation should be triggered.

This behavior resulted from the coupling of two operations within the same ROS callback function, the update of the Supervisor's internal variables for the robot that sent the state message, and the advancement of the state machines responsible for monitoring the execution progress of the robots. As a result, each individual state update could immediately influence the global replanning logic.

The progress of each robot was represented by its current position along the assigned path, typically expressed in terms of the number of completed path steps and the percentage of progression within the current step. If the difference between the progress of two or more robots exceeded a predefined threshold, the Supervisor interpreted this as a coordination inconsistency and triggered a new replanning operation.

However, since robot state updates were processed sequentially in the ROS callback queue, this mechanism could lead to inconsistent replanning triggers. When the Supervisor processed the state update of a given robot, the states of the remaining robots might not yet have been updated with their most recent information. As a result, the progress comparison used to decide whether replanning was necessary could be based on a partially updated system state.

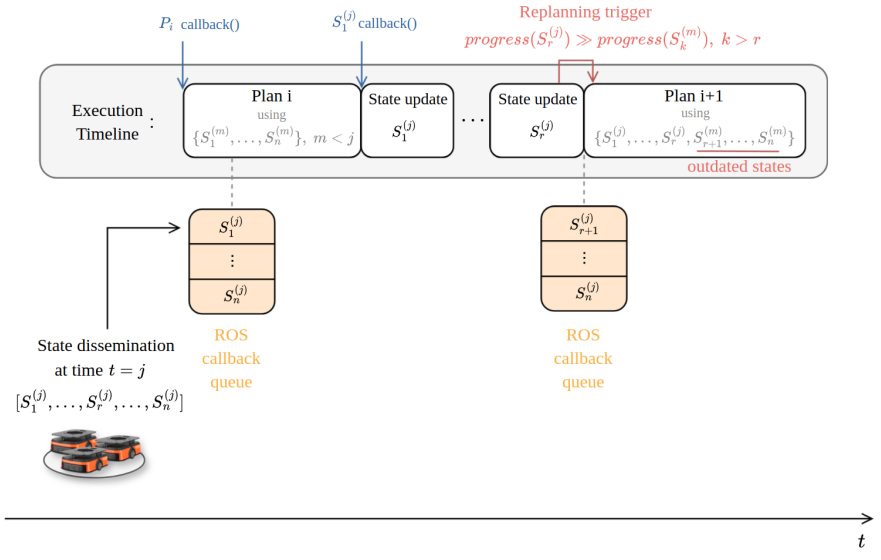


Figure 4.3: Replanning triggered from temporally inconsistent robot states.

As illustrated in Figure 4.3, Plan i is initially executed using the state set $\{S_1^{(m)}, \ldots, S_n^{(m)}\}$, corresponding to an earlier instant $t = m$. At a later instant $t = j$, a new state dissemination occurs, producing updated state messages $\{S_1^{(j)}, \ldots, S_n^{(j)}\}$ for the robots.

For instance, after processing the callback associated with robot 1, the internal state of that robot is updated to $S_1^{(j)}$, while the states of the remaining robots may still correspond to older values. Later, when the callback associated with robot r is processed, the Supervisor evaluate the replanning condition by comparing the progress of robot r with the progress of the remaining robots. This evaluation may trigger a replanning operation if the progress difference exceeds the predefined threshold, represented in Figure 4.3 as $progress(S_r^{(j)}) \gg progress(S_k^{(m)})$, with $k > r$. However, at that moment, only part of the state dissemination has been processed. As a result, the new plan, denoted as Plan $i+1$, may be computed using a mixed state set composed of updated states for the robots whose callbacks have already been processed, $\{S_1^{(j)}, \ldots, S_r^{(j)}\}$, and outdated states for the remaining robots, $\{S_{r+1}^{(m)}, \ldots, S_n^{(m)}\}$.

This behavior can introduce two related problems:

1. The decision to trigger replanning may be affected by temporally inconsistent information, since the progress of one robot is compared against progress values from other robots that may still refer to older states, wich can lead to unnecessary replanning operations.

2. If replanning is triggered immediately after processing an state update, the planner may be executed using robot states associated with different points in time. In this case, the generated plan is not based on a consistent global snapshot of the multi-robot system, but rather on a combination of updated and outdated agent states.

These limitations motivated the introduction of a batching mechanism, described in the following subsection, to decouple message reception from immediate planning and replanning decisions.

4.2.3 Proposed Batching Mechanism

To mitigate the temporal inconsistencies identified in the previous subsections, a batching mechanism was introduced for both planning requests and robot state updates. The main objective of this modification was to prevent the Supervisor from triggering planning or replanning operations based on individual messages processed in isolation.

Instead of forwarding each planning request directly to the Path Planner, the system first aggregates requests received close in time. Similarly, robot state updates are collected and processed together before the Supervisor evaluates whether replanning is required. This approach reduces redundant planner executions and allows replanning decisions to be based on a more temporally consistent representation of the multi-robot system.

The proposed architecture introduces two intermediate components between the robots and the Supervisor: the Robot Request Dispatcher and the Robot State Dispatcher. The Robot Request Dispatcher collects planning requests from multiple robots and forwards them to the Supervisor as a single batch. The Robot State Dispatcher performs an analogous role for robot state updates, ensuring that multiple state messages are processed together before progress comparisons are evaluated.

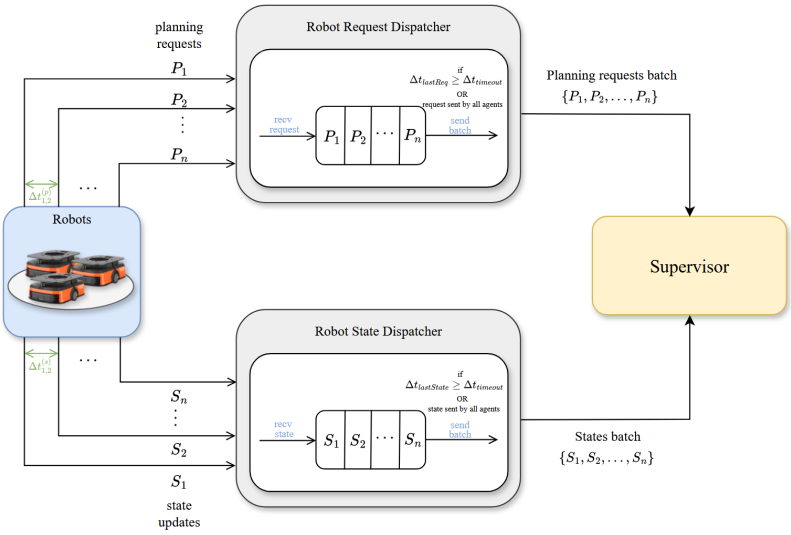


Figure 4.4: Proposed coordination architecture with batching mechanisms for planning requests and robot state updates.

As illustrated in Figure 4.4, planning requests are no longer processed independently as soon as they arrive at the Supervisor. Instead, the Robot Request Dispatcher accumulates incoming requests until one of two conditions is satisfied:

1. Planning requests have been received from all robots in the system, ensuring that the complete set of requests is available for the next planning operation;
2. A predefined inactivity timeout, denoted as T_{timeout}, expires after the most recent planning request and is restarted whenever a new planning request is received.

A similar strategy is applied to robot state updates through the Robot State Dispatcher. In contrast to the baseline implementation, with the presented mechanism, the Supervisor can update the internal states of all robots represented in the batch before advancing the corresponding state machines and comparing robot progress values. This reduces the likelihood of evaluating replanning conditions using a mixture of recently updated and outdated states. Consequently, replanning decisions are based on a more coherent approximation of the global system state.

Overall, the batching mechanism introduces a controlled delay before processing planning requests and robot state updates. This delay is bounded by the timeout parameter, which must balance batch completeness and response latency. In this context, the bounded delay introduced by batching is preferable to the unbounded accumulation of redundant planner executions observed in the baseline implementation, while improving the temporal consistency of the planning pipeline.

4.2.4 Effect of the Batching Mechanism

The effect of the proposed batching mechanism is illustrated in Figure 4.5. In contrast to the baseline behavior, planning requests generated within a short time interval are first aggregated by

the Robot Request Dispatcher before being forwarded to the Supervisor. As a result, the requests $\{P_1, \ldots, P_n\}$ are processed as a single batch, producing only one planner execution instead of a sequence of redundant planning processes.

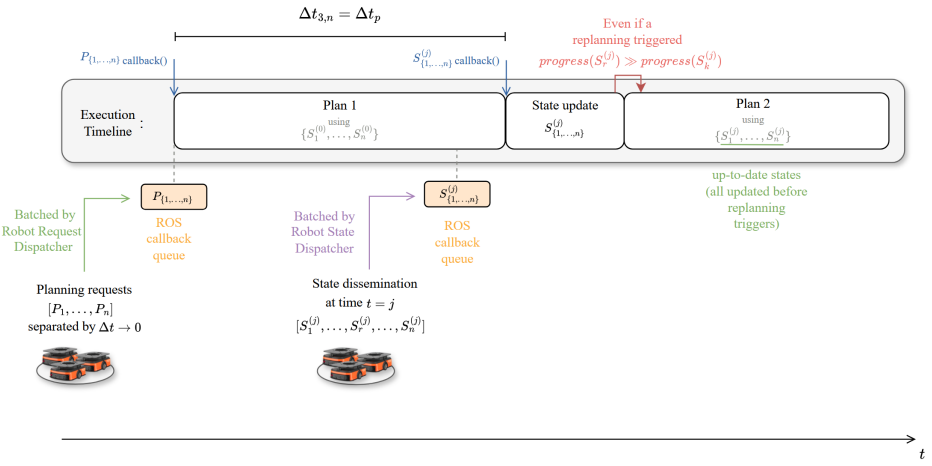


Figure 4.5: Execution flow after introducing batching mechanisms for planning requests and robot state updates.

As shown in the figure, the batched planning request triggers a single planning process, denoted as Plan 1, computed using the state set $\{S_1^{(0)}, \ldots, S_n^{(0)}\}$. Since the requests $\{P_1, \ldots, P_n\}$ are aggregated before being forwarded to the Supervisor, they are processed as a single planning event rather than as a sequence of independent callbacks. As a result, the $n-1$ intermediate planner executions observed in the baseline implementation are avoided.

The batching mechanism introduces a delay before the batch is forwarded to the Supervisor, denoted as Δt_{batch}. Since the inactivity timeout is restarted whenever a new request arrives, Δt_{batch} depends on the temporal distribution of incoming requests. In the worst case, if each new request arrives immediately before the inactivity timeout expires and the batch is completed only when the last robot sends its request, this delay is bounded by:

$$\Delta t_{\text{batch}} \leq (n-1) T_{\text{timeout}} \tag{4.5}$$

Nevertheless, T_{timeout} is selected to be significantly smaller than the planner execution time, making this additional batching delay small in relation to the computational cost of planning. Once the batch is forwarded, the dominant remaining delay is the execution time of a single planning process, denoted by Δt_p.

Assuming that message transmission delays remain negligible in the local simulation environment, the total response time between the first planning request and the corresponding plan delivery can therefore be approximated as:

$$\Delta t^{\text{batch}}_{\text{response}} \approx \Delta t_{\text{batch}} + \Delta t_p \tag{4.6}$$

This response time contrasts with the baseline behavior described in Equation 4.4, where the response time for the final planning process could accumulate the execution time of multiple redundant planner calls.

A similar improvement is obtained for robot state updates. When a new state dissemination occurs at time $t = j$, the updated states $\{S_1^{(j)}, \ldots, S_n^{(j)}\}$ are first aggregated by the Robot State Dispatcher and then forwarded to the Supervisor as a single batch.

With batching, state updates accumulated before the inactivity timeout expires are incorporated before the progress comparison is performed. Therefore, if replanning is triggered, the new plan is computed using a more coherent and up-to-date state set, $\{S_1^{(j)}, \ldots, S_n^{(j)}\}$, rather than a state representation made inconsistent by partially processed callback messages.

This mechanism therefore introduces a trade-off between temporal consistency and response latency. Since messages are accumulated before being forwarded to the Supervisor, planning requests and robot state updates are no longer processed immediately after reception. In the considered system, this additional delay is not expected to compromise coordination performance, since robot states are published at a higher frequency than that required by the Supervisor to make coordination decisions. If response latency became critical, more recent robot states could be estimated through interpolation between consecutive updates. However, this mechanism was not required in the evaluated system.

4.3 Summary

This chapter presented the stabilization process applied to the baseline coordination pipeline before effectively improving the Path Planner. The main issues identified were redundant planner executions and replanning decisions based on temporally inconsistent robot states, both resulting from the sequential processing of messages in the ROS callback queue.

The analysis showed that nearly simultaneous planning requests could trigger multiple consecutive planner executions, increasing response time and delaying the processing of updated robot states. It also showed that processing robot state updates individually could lead to replanning decisions based on mixed state snapshots, composed of updated states for some robots and outdated states for others.

To mitigate these issues, batching mechanisms were introduced for planning requests and robot state updates. Planning request batching reduces multiple nearly simultaneous planner calls to a single execution, while state update batching allows replanning conditions to be evaluated after incorporating a more coherent set of robot states. Although this mechanism introduces a bounded delay controlled by the timeout parameter, it improves the temporal consistency and reliability of the baseline system, providing a stable foundation for the planner improvements evaluated in the following chapters.

Chapter 5

Prioritization Heuristics Design and Development

After stabilizing the baseline coordination pipeline, the work focused on improving the TEA* planner through priority ordering. Due to the sequential nature of TEA*, each planned path constrains the following ones, making the selected ordering directly affect solution cost, delays, and feasibility.

This chapter presents the prioritization heuristics developed to generate alternative robot orderings for TEA*. Section 5.1 introduces the motivation and design objectives, while Section 5.2 presents the common score-based formulation. The remaining sections describe the proposed heuristic families, namely roadmap distance, topology, surroundings, conflict-based, and replanning heuristics, including their scoring criteria, ordering variants, algorithms, and computational costs.

These heuristics provide the alternative priority orderings later explored in parallel in the following chapter.

5.1 Motivation and Design Objectives

The motivation for developing prioritization heuristics arises from the fact that the same planning instance may produce substantially different results depending only on the order in which robots are planned. In constrained environments, assigning priority to one robot may reduce its own planning difficulty while increasing the constraints imposed on the remaining robots.

Since the best ordering depends on robot distribution, goals, roadmap structure, and path interactions, no single prioritization rule is expected to perform well in all scenarios. For this reason, the objective of this work is not to identify one dominant heuristic, but to design a diverse set of complementary ordering strategies. These heuristics should produce priority orderings that are informative enough to improve planning feasibility or solution quality, while remaining computationally lightweight when compared with the execution time of TEA*. Given that prioritization is performed before planner execution, excessive heuristic computation time could reduce or even eliminate the practical gains obtained from improved orderings.

5.2 Heuristic Design Methodology

The heuristics developed in this work follow a common score-based formulation. Given a set of robots

$$R = \{r_1, r_2, \ldots, r_n\} \tag{5.1}$$

each heuristic assigns a priority score, $score_i$, to every robot, r_i, according to the information available in the current planning instance. For a given heuristic, the priority score assigned to robot r_i is denoted as:

$$score_i = h(d_i) \tag{5.2}$$

where d_i denotes the input data used by the heuristic to evaluate robot r_i.

Depending on the heuristic being applied, d_i may include information such as the robot's current position, target vertex, estimated path, predicted conflicts, roadmap-related properties, or data from previous coordination cycles.

After computing the scores, the priority ordering is obtained using a common sorting procedure. The sorting direction is defined by the parameter *mode*. In the *First* mode, robots with higher scores are planned earlier, while in the *Last* mode, robots with lower scores are planned earlier. This operation is denoted as

$$\pi = \textsc{SortByScore}(R, score, mode), \tag{5.3}$$

, where π represents the resulting priority permutation of the robots.

The developed heuristics are organized into five main groups according to the type of information they exploit:

- **Roadmap distance heuristics**, based on distances or estimated path lengths in the roadmap graph;
- **Topological heuristics**, based on structural properties of the graph and its constrained regions;
- **Surroundings-based heuristics**, based on the spatial distribution of robots around their current positions or target goals;
- **Conflict-based heuristics**, based on estimated interactions between tentative robot paths;
- **Replanning heuristics**, based on information from previous plans or coordination cycles.

Together, these groups cover complementary dimensions of the coordination problem, allowing the proposed heuristics to address robot prioritization from different perspectives. Each group is examined in the following sections, where the corresponding heuristics are described in terms of their motivation, scoring procedure, and computational implications.

5.3 Roadmap Distance Heuristics

Roadmap distance heuristics define robot priority orderings using measures related with distance between each robot and its target. The main intuition behind this family is that robots associated with longer movements may have a higher probability of interacting with other robots, since their paths tend to cover a larger portion of the environment and may traverse more shared resources.

These heuristics have the advantage of being computationally lightweight, as the priority permutation is computed from simple distance measures or shortest-path estimates. However, they do not explicitly account for path conflicts, local robot density, or topological constraints. Instead, they rely on travel distance as a simple indicator of the relative difficulty of planning for each robot.

In this work, two criteria based on distance are considered, namely the distance between each robot and its target and the length of an estimated path in the roadmap graph. These criteria give rise to the *Furthest Robot* and *Longest Path* heuristics, respectively.

5.3.1 Furthest Robot

The *Furthest Robot* heuristic assigns priorities according to the direct distance between each robot and its target.

For each robot r_i, let $s_i = (x_i, y_i)$ denote its current position and let $g_i = (x_{g_i}, y_{g_i})$ denote its target position. The score assigned to robot r_i is computed as the Euclidean distance between these two points:

$$score_i = dist(s_i, g_i) = \|g_i - s_i\|_2 \tag{5.4}$$

Figure 5.1 illustrates this computation for a single robot and target pair. In the example, the score assigned to robot r_1 corresponds to the direct distance between its current position, s_1, and its target position, g_1.

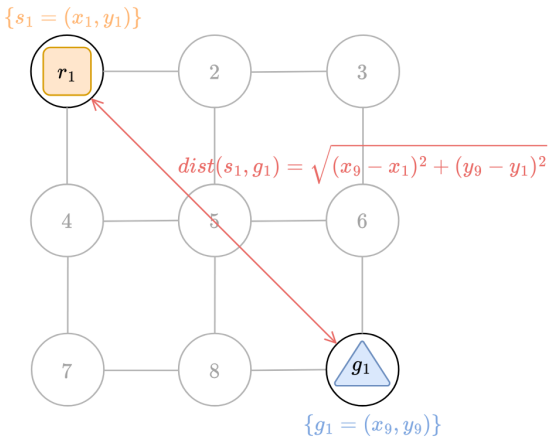


Figure 5.1: Score computation for the Furthest Robot heuristic.

5.3.1.1 Ordering variants

Two ordering variants were considered. In the *Furthest Robot First* variant, robots with higher distance scores are planned earlier, following the intuition that longer movements should be planned before the search space becomes more constrained.

In the *Furthest Robot Last* variant, the ordering is reversed, and robots with lower distance scores are planned earlier. This strategy may be useful in scenarios where the objective is to maximize the number of robots that successfully reach their targets, especially when a complete solution for all robots is difficult or infeasible. By prioritizing robots closer to their goals, the planner may allow a larger subset of robots to complete their paths before the remaining planning problem becomes too constrained.

5.3.1.2 Core Algorithm

The procedure is summarized in Algorithm 1. For each robot, the heuristic computes the Euclidean distance between its current roadmap position and its target vertex, and then applies the sorting procedure to obtain the final priority ordering.

Algorithm 1 Furthest Robot Heuristic

Require: Set of robots R, current closest-vertex positions s_i on the roadmap, current target vertices g_i on the roadmap, ordering mode $mode$

Ensure: Priority ordering π

1: **for** each robot $r_i \in R$ **do**
2: $\quad score_i \leftarrow dist(s_i, g_i)$
3: **end for**
4: $\pi \leftarrow \textsc{SortByScore}(R, score, mode)$
5: **return** π

In the following complexity analyses, $n = |R|$ denotes the number of robots. Each score is computed in constant time, resulting in $O(n)$ time for all robots. The final score-based sorting step requires $O(n \log n)$ time. Thus, the total computational cost is

$$O(n) + O(n \log n) \tag{5.5}$$

which is dominated by the sorting step.

5.3.2 Longest Path

The *Longest Path* heuristic assigns priorities according to the cost of an estimated shortest path on the roadmap graph. Unlike the *Furthest Robot* heuristic, which uses only the Euclidean distance between the current and target vertices, this heuristic accounts for the actual connectivity of the roadmap. This is particularly relevant in scenarios where the Euclidean distance does not reflect

the real distance that the robot must travel, for example when obstacles force the robot to follow a longer route through the graph.

For each robot r_i, a single-robot shortest path is computed between its current vertex s_i and its target vertex g_i using A^*. The resulting path is represented as

$$P_i^* = (v_1, v_2, \ldots, v_k) \tag{5.6}$$

with $v_1 = s_i$ and $v_k = g_i$. The priority score of robot r_i is then defined as the total cost of the edges along this path:

$$score_i = \sum_{\ell=1}^{k-1} w_{v_\ell, v_{\ell+1}} \tag{5.7}$$

where $w_{v_\ell, v_{\ell+1}}$ denotes the weight of the edge connecting consecutive vertices v_ℓ and $v_{\ell+1}$.

In the example shown in Figure 5.2, the path computed for robot r_1 traverses the sequence of vertices from its initial position to its goal. The corresponding score is obtained by summing the costs of the selected roadmap edges.

$$score_1 = w_{1,2} + w_{2,5} + w_{5,6} + w_{6,9} \tag{5.8}$$

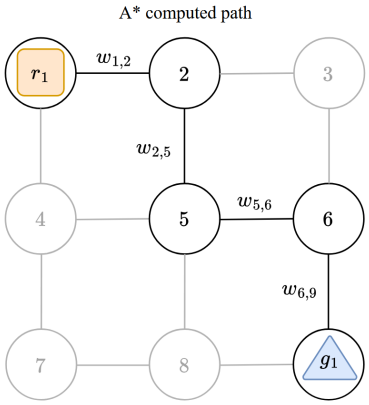


Figure 5.2: Illustration of the Longest Path heuristic.

5.3.2.1 Ordering variants

Following the same logic used for the *Furthest Robot* heuristic, two ordering variants were considered. In the *Longest Path First* variant, robots with higher path-cost scores are planned earlier, while in the *Longest Path Last* variant, robots with lower path-cost scores are planned earlier. Both variants use the same priority score definition and differ only in the sorting direction.

5.3.2.2 Core Algorithm

Algorithm 2 summarizes the *Longest Path* heuristic. For each robot, the algorithm computes a single-agent shortest path on the roadmap, uses the corresponding path cost as the priority score,

and then applies the generic score-based sorting procedure to obtain the final ordering.

Algorithm 2 Longest Path Heuristic

Require: Set of robots R, current positions s_i, target positions g_i, ordering mode *mode*
Ensure: Priority ordering π
1: **for** each robot $r_i \in R$ **do**
2: $p_i \leftarrow A^*(s_i, g_i)$ ▷ single-robot shortest path on the roadmap
3: $score_i \leftarrow cost(p_i)$ ▷ sum of edge costs along p_i
4: **end for**
5: $\pi \leftarrow$ SORTBYSCORE$(R, score, mode)$
6: **return** π

The dominant operation in this heuristic is the computation of one A^* path per robot. As discussed in Section 2.3.1.2, in the worst case, A^* may explore the same search space as Dijkstra's algorithm. Therefore, using the complexity given in Equation 2.3, computing the priority scores for all robots requires $O\big(n \cdot (|V| + |E|) \log |V|\big)$ time. Since the final score-based sorting step requires $O(n \log n)$ time, the total computational cost is

$$O\big(n \cdot (|V| + |E|) \log |V|\big) + O(n \log n) \tag{5.9}$$

which is dominated by the n shortest-path computations.

5.4 Topological Heuristics

Topological heuristics aim to capture the structural constraints imposed by the roadmap graph on the path of each robot. Unlike distance-based heuristics, which assign priorities according to geometric or path-length information, this family of heuristics focuses on the connectivity of the roadmap and on the amount of movement flexibility available along each robot path.

To quantify this notion of local flexibility, the concept of vertex Degrees of Freedom (DoF) is considered. In this context, the DoF of a roadmap vertex corresponds to the number of possible movements available from that vertex. Each roadmap vertex v is associated with a set of directly connected neighbouring vertices, denoted by $N(v)$. The DoF of v is defined as

$$\mathrm{DoF}(v) = |N(v)| \tag{5.10}$$

where $|N(v)|$ represents the number of neighbours of vertex v.

For each robot r_i, the topological heuristics first compute a single-robot shortest path between its current vertex s_i and its target vertex g_i using A^*. The resulting path is denoted by

$$P_i^* = (v_1, v_2, \ldots, v_k) \tag{5.11}$$

with $v_1 = s_i$ and $v_k = g_i$. The vertices belonging to this path are then evaluated according to their DoF values.

Vertices with lower DoF values typically correspond to more constrained regions of the environment, such as corridors. In these areas, robots have fewer alternative movements available, which can make coordination more difficult when several robots need to traverse the same region. Conversely, vertices with higher DoF values tend to represent more open areas of the roadmap, where robots have more possible movement choices and greater flexibility to avoid conflicts. Figure 5.3 illustrates this concept on a roadmap path computed with A^*.

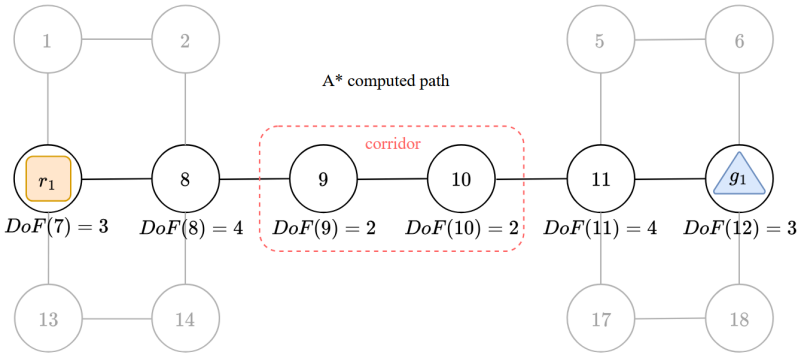


Figure 5.3: Illustration of DoF values along a roadmap path.

Based on this information, the heuristics in this family evaluate the roadmap path associated with each robot according to the topological properties of the vertices it traverses. In particular, they aim to identify paths that pass through structurally constrained regions, such as low-DoF areas, and distinguish them from paths that remain mostly in more open and flexible parts of the graph.

The main intuition is that robots crossing low-DoF regions are more sensitive to space-time reservations introduced by previously planned robots. Since these regions offer fewer alternative movements, delaying the planning of such robots may increase the likelihood of conflicts or infeasible plans.

5.4.1 Average DoF Path

The *Average DoF* heuristic assigns priorities according to the average DoF of the vertices belonging to the roadmap path of each robot. Therefore, it provides an estimate of whether the robot is expected to move mostly through open regions of the roadmap or through more constrained areas.

The priority score of robot r_i is defined as

$$score_i = \frac{1}{|P_i^*|} \sum_{v \in P_i^*} \mathrm{DoF}(v). \tag{5.12}$$

In the example shown in Figure 5.3, the path computed for robot r_1 includes vertices 7, 8, 9, 10, 11, and 12. The corresponding score is obtained by averaging the DoF values of these vertices.

$$score_1 = \frac{\mathrm{DoF}(7) + \mathrm{DoF}(8) + \mathrm{DoF}(9) + \mathrm{DoF}(10) + \mathrm{DoF}(11) + \mathrm{DoF}(12)}{6} = 3 \tag{5.13}$$

5.4.1.1 Ordering variants

Two ordering variants were considered. In the *High Average DoF First* variant, robots with higher average DoF scores are planned earlier, prioritizing paths that traverse more open regions of the roadmap. In the *Low Average DoF First* variant, the ordering is reversed, and robots with lower average DoF scores are planned earlier, prioritizing paths that traverse more constrained regions. Both variants use the same score and differ only in the sorting direction.

5.4.1.2 Core Algorithm

Algorithm 3 summarizes the *Average DoF* heuristic. For each robot, the algorithm computes an A^* path, accumulates the DoF values of the vertices along that path and uses their average as the priority score.

Algorithm 3 Average Path DoF Heuristic

Require: Set of robots R, roadmap graph G, current positions s_i, target positions g_i, ordering mode *mode*

Ensure: Priority ordering π

for each robot $r_i \in R$ **do**
  $p_i \leftarrow A^*(s_i, g_i)$ ▷ single-robot shortest path on the roadmap
  $N_i \leftarrow$ distinct vertices along p_i
  $DoFsum_i \leftarrow 0$
  for each vertex $v \in N_i$ **do**
    $N_v \leftarrow \emptyset$ ▷ set of distinct neighbours of v
    for each edge e in AdjacencyList(v) **do**
      $u \leftarrow$ OppositeEndpoint(e, v) ▷ neighbour reached through edge e
      insert u into N_v
    end for
    $\mathrm{DoF}(v) \leftarrow |N_v|$ ▷ number of distinct neighbours of v
    $DoFsum_i \leftarrow DoFsum_i + \mathrm{DoF}(v)$
  end for
  $score_i \leftarrow DoFsum_i / |N_i|$
end for
$\pi \leftarrow$ SortByScore($R, score, mode$)
return π

The dominant operations in this heuristic are the single-robot path searches and the evaluation of the DoF values along the resulting paths. Using the A^* worst-case complexity given in Equation 2.3, the path computation step is applied once per robot.

After each path is obtained, the heuristic visits its vertices and computes the DoF of each one by checking its adjacency list. If L_i is the number of edges in the path of robot r_i, and $\Delta = \max_{v \in V} \mathrm{DoF}(v)$ is the maximum degree of any roadmap vertex, the DoF aggregation step costs at most $O(L_i \cdot \Delta)$ for that robot. Considering all robots, this cost is upper-bounded by

$$O(n \cdot L_{\max} \cdot \Delta) \tag{5.14}$$

where $L_{\max} = \max_i L_i$. The final score-based sorting step requires $O(n \log n)$ time. Thus, the total computational cost is given by

$$nO\big(n \cdot (|V| + |E|) \log |V|\big) + O\big(n \cdot L_{\max} \cdot \Delta\big) + O(n \log n). \tag{5.15}$$

5.4.2 Low DoF Sum

The *Low DoF Sum* heuristic assigns priorities according to the number of constrained vertices traversed by the roadmap path of each robot. Instead of averaging the DoF values along the path, as in the *Average DoF Path* heuristic, this approach explicitly counts how many vertices are classified as topologically restricted.

A vertex is considered constrained when its DoF is lower than or equal to a predefined threshold τ. The priority score of robot r_i is defined as

$$score_i = \sum_{v \in P_i^*} \delta(v), \tag{5.16}$$

with

$$\delta(v) = \begin{cases} 1, & \mathrm{DoF}(v) \leq \tau, \\ 0, & \text{otherwise.} \end{cases} \tag{5.17}$$

Therefore, the score corresponds to the number of low-DoF vertices traversed by the path of the robot.

Compared with the *Average DoF Path* heuristic, the *Low DoF Sum* heuristic provides a more explicit measure of path restriction. While *Average DoF Path* summarizes the overall topological flexibility of a path through a mean value, *Low DoF Sum* directly counts the number of vertices that satisfy a predefined restriction criterion. As a result, it can better emphasize paths that cross several clearly constrained regions, even when the average DoF of the complete path is not particularly low.

This distinction can lead to different priority scores in scenarios where the distribution of constrained vertices along the path is relevant. For example, two paths may have similar average DoF values, but one may traverse several very restricted vertices while the other only passes through

moderately constrained areas. In such cases, the *Low DoF Sum* heuristic tends to penalize the first path more strongly, whereas the average-based score may smooth out this difference.

However, this explicitness also introduces a limitation. The heuristic depends directly on the threshold τ, which defines what is considered a low-DoF vertex. Therefore, its behavior is more sensitive to parameter selection and to the specific connectivity characteristics of the roadmap. In contrast, the *Average DoF Path* heuristic is more general, since it does not require a hard classification of vertices as constrained or unconstrained.

Considering the path illustrated in Figure 5.3, the threshold is set to $\tau = 2$. Therefore, only vertices with $\mathrm{DoF}(v) \leq 2$ are classified as constrained. In this case, vertices 9 and 10 satisfy the restriction condition, while vertices 7, 8, 11, and 12 do not. The Low DoF Sum score for robot r_1 is therefore:

$$\mathrm{score}(r_1) = \delta(7) + \delta(8) + \delta(9) + \delta(10) + \delta(11) + \delta(12) = 0+0+1+1+0+0 = 2. \quad (5.18)$$

5.4.2.1 Core Algorithm

Algorithm 4 follows the same structure as Algorithm 3, differing only in the score computation.

Algorithm 4 Low DoF Sum Heuristic

Require: Set of robots R, roadmap graph G, current positions s_i, target positions g_i, ordering mode *mode*, DoF threshold τ

Ensure: Priority ordering π

for each robot $r_i \in R$ **do**
  $p_i \leftarrow A^*(s_i, g_i)$ ▷ single-agent shortest path on the roadmap
  $N_i \leftarrow$ distinct vertices along p_i
  $score_i \leftarrow 0$
  for each vertex $v \in N_i$ **do**
    $N_v \leftarrow \emptyset$ ▷ set of distinct neighbours of v
    for each edge e in AdjacencyList(v) **do**
      $u \leftarrow$ OppositeEndpoint(e, v) ▷ neighbour reached through edge e
      insert u into N_v
    end for
    $\mathrm{DoF}(v) \leftarrow |N_v|$ ▷ number of distinct neighbours of v
    if $\mathrm{DoF}(v) \leq \tau$ **then**
      $score_i \leftarrow score_i + 1$ ▷ count constrained vertex
    end if
  end for
end for
$\pi \leftarrow$ SortByScore$(R, score, mode)$
return π

This heuristic has the same computational complexity as the *Average DoF Path* heuristic. The only difference lies in the aggregation step, since *Low DoF Sum* counts the vertices that satisfy $\mathrm{DoF}(v) \leq \tau$ instead of averaging the DoF values. Therefore, the total computational cost remains essentially the same as in Equation 5.15.

5.5 Surroundings Heuristics

Surroundings heuristics characterize each robot according to the spatial distribution of the other robots around a selected reference point. Unlike the previous heuristic families, which mainly focus on individual distance, path length, or topological properties of a robot path, this family uses information about how robots are distributed relative to one another. The objective is to assign priorities based on the local interaction potential induced by nearby robots.

In this work, the selected reference point can correspond either to the current position of the robot or to its target position. When the current position is used, the heuristic evaluates how crowded the region around the robot is at the beginning of the planning process. Robots starting in dense regions may have more difficulty initiating their movements, since several agents may need to leave nearby roadmap areas at similar times. When the target position is used instead, the heuristic evaluates the concentration of goals. Robots whose goals lie in dense regions may be more likely to compete for access to the same final area or to nearby roadmap resources.

Therefore, surroundings heuristics provide a complementary prioritization criterion based on the local spatial context of each robot. Depending on the specific heuristic, this context can be measured using different notions of neighbourhood, such as Euclidean distance, graph-based proximity, or cluster membership. This allows the planner to distinguish robots associated with crowded regions from those that are more spatially isolated, either at their initial positions or at their goals.

5.5.1 Radius Neighbor Density

The Radius Neighbour Density heuristic assigns priorities according to the local density of robots around a selected reference point. For each robot, the heuristic counts how many other robots are located within a predefined distance threshold. This provides a simple proximity-based estimate of how crowded the surrounding region is.

Considering that n_i denotes the reference point associated with robot r_i, which may correspond either to its current position or to its target position, depending on the variant being evaluated. Given a distance threshold R_{th}, the score of robot r_i is defined as:

$$score_i = \sum_{\substack{r_j \in R \\ j \neq i}} \rho(i, j), \tag{5.19}$$

where

$$\rho(i,j) = \begin{cases} 1, & d(n_i, n_j) \leq R_{\text{th}}, \\ 0, & \text{otherwise.} \end{cases} \tag{5.20}$$

Here, $d(n_i, n_j)$ represents the Euclidean distance between the reference points associated with robots r_i and r_j. Therefore, the score corresponds to the number of neighbouring robots located within radius R_{th} of robot r_i.

Robots with higher scores are located in denser local regions, since more robots are found within their neighbourhood. When the current position is used as the reference point, this may indicate robots that could have more difficulty initiating their movements due to the presence of nearby agents. When the target position is used instead, higher scores indicate goals located in more concentrated areas, where several robots may compete for access to nearby roadmap resources.

This heuristic is strongly affected by the choice of the radius threshold. If R_{th} is too small, most robots may have few or no neighbours, making the heuristic unable to distinguish between different local configurations. Conversely, if R_{th} is too large, many robots may be counted as neighbours, reducing the locality of the measure. Therefore, the value of R_{th} directly controls the scale at which local density is evaluated.

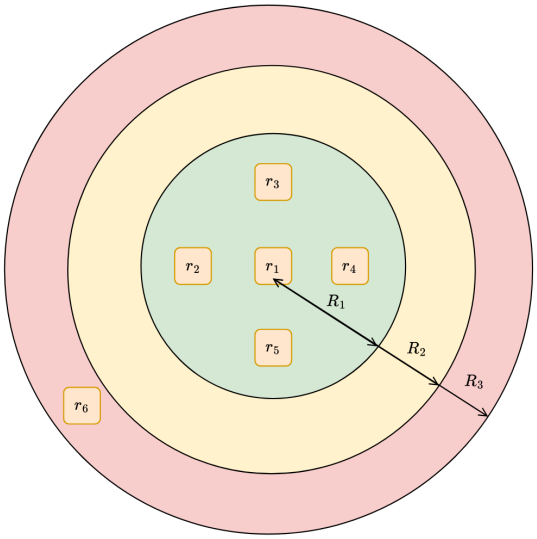


Figure 5.4: Illustration of the Radius Neighbour Density heuristic.

For the configuration illustrated in Figure 5.4, the score of robot r_1 varies with the selected radius:

$$score_1 = \begin{cases} 4, & R_{\text{th}} = R_1, \\ 4, & R_{\text{th}} = R_2, \\ 5, & R_{\text{th}} = R_3. \end{cases} \tag{5.21}$$

This illustrates the direct dependence of the heuristic on the chosen radius threshold.

5.5.1.1 Ordering variants

Two ordering variants were considered for the *Radius Neighbour Density* heuristic. The first one, *High Radius Density Goals First*, computes the density score using the target positions of the robots and prioritizes robots whose goals are located in denser regions. This strategy is motivated by the behavior of TEA* in scenarios where several robots have nearby goals. In such cases, planning first the robots that must reach the most crowded goal regions may help avoid situations where robots assigned to peripheral goals introduce reservations that later block access to the denser central area.

The second variant, *Low Radius Density Positions First*, computes the density score using the initial positions of the robots and prioritizes robots that start in less crowded regions. This follows the intuition that, when robots are initially clustered, it may be beneficial to first move the robots located closer to the periphery of the cluster. By freeing these outer positions earlier, the planner may create more space for the robots located in more central and constrained starting regions to find feasible paths afterwards.

Therefore, these two variants use the same radius-density score but apply it to different reference points and with opposite sorting directions.

5.5.1.2 Core Algorithm

Algorithm 5 summarizes the *Radius Neighbour Density* heuristic. The heuristic is applied to a reference point n_i, which may correspond either to the current position or to the target position of robot r_i, depending on the selected variant.

Algorithm 5 Radius Neighbour Density Heuristic

Require: Set of robots R, reference function $\mathrm{ref}(\cdot)$, radius threshold R_{th}, ordering mode *mode*
Ensure: Priority ordering π

for each robot $r_i \in R$ **do**
  $n_i \leftarrow \mathrm{ref}(r_i)$ ▷ current position or target position
  $score_i \leftarrow 0$
  for each robot $r_j \in R$ with $j \neq i$ **do**
    $n_j \leftarrow \mathrm{ref}(r_j)$
    if $d(n_i, n_j) \leq R_{\mathrm{th}}$ **then**
      $score_i \leftarrow score_i + 1$ ▷ count nearby robot
    end if
  end for
end for
$\pi \leftarrow \textsc{SortByScore}(R, score, mode)$
return π

The heuristic computes each density score by comparing the reference point of a robot with the reference points of all remaining robots and counting how many lie within the radius threshold R_{th}.

Since every robot is compared with the remaining $n-1$ robots, computing all density scores requires $O(n(n-1)) \approx O(n^2)$ pairwise checks. The final score-based sorting step requires $O(n \log n)$ time. Thus, the total computational cost is given by

$$O(n^2) + O(n \log n). \tag{5.22}$$

It is also important to note that this complexity does not scale with the value of R_{th}. The radius only affects the outcome of each pairwise comparison, not the number of comparisons performed.

5.5.2 K-hop Neighbor Density

The *K-hop Neighbour Density* heuristic assigns priorities according to the number of robots located within a graph-based neighbourhood of each robot. Unlike the *Radius Neighbour Density* heuristic, which defines proximity using a geometric distance threshold, this heuristic defines neighbourhoods directly from the roadmap connectivity.

Given a hop limit k_{hops}, let $N_{k_{\text{hops}}}(n_i)$ denote the set of roadmap vertices that can be reached from n_i in at most k_{hops} graph hops. The score of robot r_i is defined as:

$$score_i = \sum_{\substack{r_j \in R \\ j \neq i}} \kappa(i, j), \tag{5.23}$$

where

$$\kappa(i, j) = \begin{cases} 1, & n_j \in N_{k_{\text{hops}}}(n_i), \\ 0, & \text{otherwise.} \end{cases} \tag{5.24}$$

Therefore, the score corresponds to the number of robots whose reference points are reachable from n_i within at most k_{hops} roadmap edges. Robots with higher scores are located in denser graph-based neighbourhoods, since more robots can be reached within the selected hop depth.

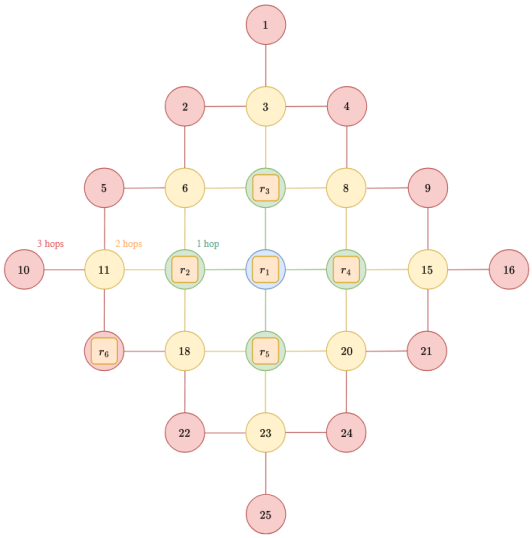


Figure 5.5: Illustration of the K-hop Neighbour Density heuristic.

For the configuration illustrated in Figure 5.5, the score of robot r_1 is:

$$score_1 = \begin{cases} 4, & k_{\text{hops}} = 1, \\ 4, & k_{\text{hops}} = 2, \\ 5, & k_{\text{hops}} = 3. \end{cases} \tag{5.25}$$

Although conceptually similar to *Radius Neighbour Density*, this heuristic defines neighbourhoods through roadmap connectivity rather than Euclidean distance. This distinction is relevant in environments with obstacles. For example, two robots may be close in continuous space but far apart on the roadmap if a wall or obstacle separates them and forces a longer route between their positions. Conversely, two robots that are farther apart in Euclidean distance may still be close in graph terms if they are connected through only a few roadmap edges.

However, its behaviour depends on the selected hop limit k_{hops}, which plays a role analogous to the radius threshold R_{th} in the previous heuristic. Small values restrict the density estimate to very local neighbourhoods, while larger values expand the considered region.

5.5.2.1 Ordering variants

The ordering variants follow the same logic as in the *Radius Neighbour Density* heuristic. *Low K-hop Density Positions First* and *High K-hop Density Goals First* differ only in the reference point used and in the sorting direction.

5.5.2.2 Core Algorithm

Algorithm 7 summarizes the *K-hop Neighbour Density* heuristic. For each robot, the roadmap is explored from the selected reference vertex using a recursive DFS K-hop expansion, and the score is computed by counting how many robot reference vertices lie within the resulting neighbourhood.

Algorithm 6 Recursive DFS K-hop Expansion

Require: Roadmap graph $G = (V, E)$, current vertex v, remaining hops h, map $bestK$
Ensure: Updated map $bestK$

```
if h ≤ 0 then
    return
end if
h ← h − 1
for each neighbour u ∈ N(v) do
    if u ∉ bestK or h > bestK[u] then
        bestK[u] ← h                          ▷ best remaining-hop value found for u
        RecursiveKHop(G, u, h, bestK)
    end if
end for
```

Algorithm 7 k-Hop Neighbour Density Heuristic

Require: Set of robots R, roadmap graph $G = (V, E)$, reference function $\text{ref}(\cdot)$, hop limit k_{hops}, ordering mode *mode*

Ensure: Priority ordering π

$robotCountAtVertex \leftarrow \emptyset$
for each robot $r_j \in R$ **do**
  $n_j \leftarrow \text{ref}(r_j)$ ▷ current position or target position
  $robotCountAtVertex[n_j] \leftarrow robotCountAtVertex[n_j] + 1$ ▷ count robots at each reference vertex
end for
for each robot $r_i \in R$ **do**
  $n_i \leftarrow \text{ref}(r_i)$ ▷ current position or target position
  $bestK \leftarrow \emptyset$
  $bestK[n_i] \leftarrow k_{\text{hops}}$
  RECURSIVEKHOP$(G, n_i, k_{\text{hops}}, bestK)$
  $N_{k_{\text{hops}}}(n_i) \leftarrow$ keys of $bestK$ ▷ vertices reachable within k_{hops} hops
  $score_i \leftarrow 0$
  for each vertex $v \in N_{k_{\text{hops}}}(n_i)$ **do**
    $score_i \leftarrow score_i + robotCountAtVertex[v]$ ▷ count robots in the k-hop neighbourhood
  end for
  $score_i \leftarrow score_i - 1$ ▷ exclude robot r_i from its own density score
end for
$\pi \leftarrow$ SORTBYSCORE$(R, score, mode)$
return π

The computational cost of the *K-hop Neighbour Density* heuristic depends on the structure of the roadmap graph, since the number of vertices reached by a k-hop expansion is determined by the graph connectivity. In this work, the analysis considers a grid-based roadmap, which is a common map representation in path-planning benchmarks and is also used in the experimental evaluation of this document. In this type of roadmap, each vertex has at most four neighbours.

Assuming an infinite 2D grid with four-neighbour connectivity, the number of vertices at exactly j hops from a reference vertex is $4j$. The number of vertices reachable within k_{hops} hops is denoted by N_k and, under this grid assumption, is given by

$$N_k = 1 + \sum_{j=1}^{k_{\text{hops}}} 4j = 1 + 4 \cdot \frac{k_{\text{hops}}(k_{\text{hops}} + 1)}{2} = 1 + 2k_{\text{hops}}(k_{\text{hops}} + 1). \tag{5.26}$$

However, in the recursive implementation, the number of reachable vertices does not necessarily match the number of vertex expansions performed by the algorithm. This occurs because a vertex may be expanded more than once if it is later reached with a larger remaining hop value. The number of vertex expansions performed during one recursive k-hop procedure is represented by X_k.

Since each reachable vertex can be expanded at most k_{hops} times, a conservative upper bound is

$$X_k \leq k_{\text{hops}} \cdot N_k. \tag{5.27}$$

This bound is conservative, since in practice many vertices are expanded fewer times, especially in finite grids and near map boundaries. Nevertheless, it provides a safe upper bound for the recursive implementation. Substituting the expression of N_k gives

$$X_k \leq k_{\text{hops}} \left(1 + 2k_{\text{hops}}(k_{\text{hops}} + 1)\right). \tag{5.28}$$

Expanding this expression results in

$$X_k \leq 2k_{\text{hops}}^3 + 2k_{\text{hops}}^2 + k_{\text{hops}}. \tag{5.29}$$

Therefore, the cost of one recursive k-hop expansion is upper-bounded by

$$O(X_k) = O\left(2k_{\text{hops}}^3 + 2k_{\text{hops}}^2 + k_{\text{hops}}\right) \tag{5.30}$$

which can be simplified to

$$O(k_{\text{hops}}^3). \tag{5.31}$$

Since one k-hop expansion is performed for each robot, the total cost of computing the density scores is

$$O(n \cdot k_{\text{hops}}^3). \tag{5.32}$$

Finally, the robots are sorted according to their scores, which requires $O(n \log n)$ time. Therefore, the total computational cost of the heuristic is

$$O(n \cdot k_{\text{hops}}^3) + O(n \log n). \tag{5.33}$$

As shown by this expression, the computational cost of the *K-hop Neighbour Density* heuristic depends directly on the selected hop limit k_{hops}. Therefore, increasing the depth of the graph exploration increases the cost of the heuristic. This contrasts with the *Radius Neighbour Density* heuristic, whose complexity does not depend on the radius threshold R_{th}, since the radius only affects the result of each pairwise comparison and not the number of comparisons performed.

Consequently, although the *K-hop Neighbour Density* heuristic may provide a density estimate that is more consistent with the roadmap structure, especially in environments with obstacles, it is expected to become less efficient as k_{hops} increases. In such cases, the radius-based heuristic may become computationally more efficient, particularly when larger neighbourhoods are considered.

5.5.3 Spatial Cluster-Based

The *Spatial Cluster-Based* heuristic groups robots according to the spatial distribution of their reference points. In the base formulation, these reference points correspond to the current positions of the robots, and the resulting heuristic corresponds to the *High Cluster Peripheral First* variant.

DBSCAN is used to obtain the spatial clusters. This choice is motivated by its density-based nature, which allows clusters to be identified without directly specifying their number beforehand. Although methods such as K-means can be combined with procedures to estimate the number of clusters, such as the elbow method, this introduces an additional indirect selection step that may not always provide a reliable criterion. DBSCAN is therefore more suitable in this context, since the number of robot groups may vary across planning instances and isolated robots should not necessarily be forced into a cluster.

After the clusters are computed, the heuristic assigns a priority score to each robot according to the size of its cluster and its relative distance to the cluster centroid. For a robot r_i belonging to cluster C_m, the centroid of the cluster is denoted by c_m, and the current position of the robot is denoted by s_i. The distance between the robot and the centroid is defined as

$$d_i = \|s_i - c_m\|_2. \tag{5.34}$$

The maximum distance between the centroid and any robot in the same cluster is defined as

$$d_{\max,m} = \max_{r_\ell \in C_m} \|s_\ell - c_m\|_2. \tag{5.35}$$

The priority score of robot r_i is then computed as

$$score_i = |C_m| \cdot \frac{d_i}{d_{\max,m}}, \qquad d_{\max,m} \neq 0 \tag{5.36}$$

where $|C_m|$ is the number of robots in cluster C_m.

The score contains two components. The factor $|C_m|$ increases the priority of robots belonging to larger clusters, reflecting the assumption that larger groups are typically more difficult to resolve in a prioritized planner. Within each cluster, the normalized distance term increases the score of robots farther from the centroid. As a result, the heuristic first favours robots in larger clusters and, inside each cluster, prioritizes those located closer to the boundary.

Figure 5.6 illustrates the scoring principle. Cluster C_1 contains more robots than cluster C_2, and therefore receives a higher cluster-size contribution. Within each cluster, the distance to the centroid determines the relative ordering of the robots according to the selected variant.

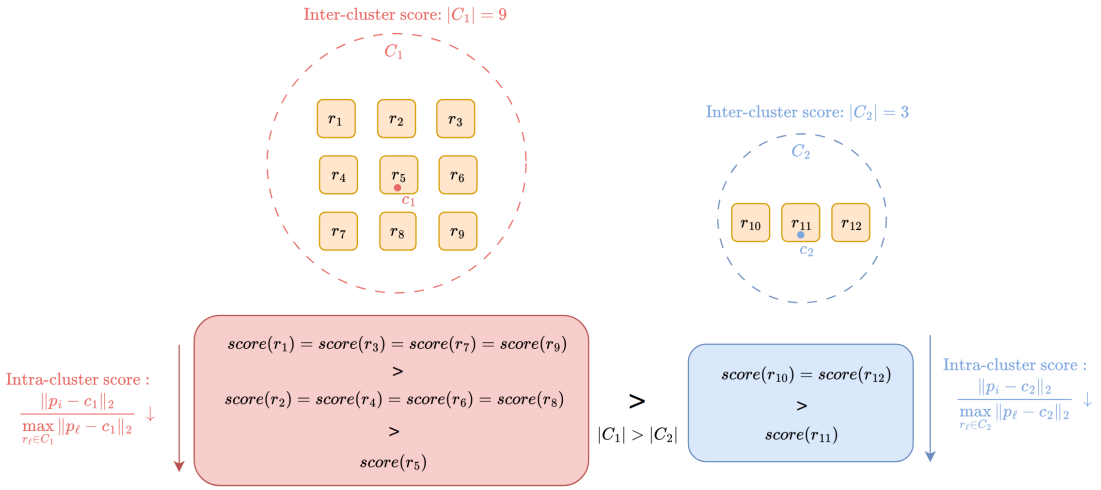


Figure 5.6: High Cluster Peripheral First scoring example.

5.5.3.1 Ordering variants

Two ordering variants were considered for the Spatial Cluster-Based heuristic. The first one is *High Cluster Peripheral First*, which corresponds to the formulation described above. This variant clusters robots according to their current positions and favours robots located farther from the centroid of larger clusters.

The second variant is *High Cluster Central Goal First*. This variant applies the same clustering principle to the target positions, but reverses the intra-cluster preference by favouring robots whose targets are closer to the centroid of their goal cluster.

For this variant, g_i represents the target position of robot r_i, and $d_i = \|g_i - c_m\|_2$ is its distance to the centroid of the corresponding goal cluster. The priority score is defined as

$$score_i = \frac{|C_m|}{d_i}, \qquad d_i \neq 0 \tag{5.37}$$

where $|C_m|$ is the size of the goal cluster.

Unlike the peripheral variant, where the score increases with the distance to the centroid, the central-goal variant assigns higher scores to robots closer to the centroid. Thus, both variants use cluster size to favour larger groups, but differ in the selected reference point and in the intra-cluster ordering criterion.

5.5.3.2 Core Algorithm

Algorithm 8 summarizes the *Spatial Cluster-Based* heuristic. The algorithm extracts the selected reference point of each robot and applies DBSCAN to identify spatial groups of nearby robots. Since DBSCAN may classify isolated points as noise, robots outside any cluster keep their initial score, set to zero in the implementation. Thus, only clustered robots are affected by the clustering component, with scores computed from the cluster size and the distance to the corresponding centroid.

Algorithm 8 Spatial Cluster-Based Heuristic

Require: Set of robots R, reference function $\mathrm{ref}(\cdot)$, score function $\mathrm{score}(\cdot)$, ordering mode *mode*
Ensure: Priority ordering π

$P \leftarrow \emptyset$ ▷ reference points used for clustering
for each robot $r_i \in R$ **do**
  $q_i \leftarrow \mathrm{ref}(r_i)$ ▷ current position or target position
  add $(q_i, \mathrm{id}(r_i))$ to P
  $score_i \leftarrow 0$
end for
$C \leftarrow \mathrm{DBSCAN}(P)$ ▷ spatial clusters of robot reference points
for each cluster $C_m \in C$ **do**
  $c_m \leftarrow$ centroid of C_m
  $d_{\max,m} \leftarrow 0$
  for each point $q_i \in C_m$ **do**
    $d_i \leftarrow \|q_i - c_m\|_2$
    $d_{\max,m} \leftarrow \max(d_{\max,m}, d_i)$
  end for
  for each point $q_i \in C_m$ associated with robot r_i **do**
    $score_i \leftarrow \mathrm{score}(|C_m|, d_i, d_{\max,m})$
  end for
end for
$\pi \leftarrow \textsc{SortByScore}(R, score, mode)$
return π

The algorithm starts by building the set of reference points, which requires processing each robot once and therefore has $O(n)$ time complexity.

The computational cost of DBSCAN depends on the implementation used to perform neighbourhood queries. In the implementation considered in this work, the ε-neighbourhood of each point is obtained by scanning all other points. Since there are at most n points, each neighbourhood query costs $O(n)$, and the complete clustering step has complexity $O(n^2)$.

After clustering, the heuristic computes the centroid of each cluster, the distance between each cluster member and the centroid, and the maximum distance inside the cluster. Since each robot belongs to at most one cluster, the sum of the cluster sizes is at most n. Therefore, the total score-computation cost over all clusters is $O(n)$.

The final score-based sorting step requires $O(n \log n)$ time. Thus, the total computational cost is given by

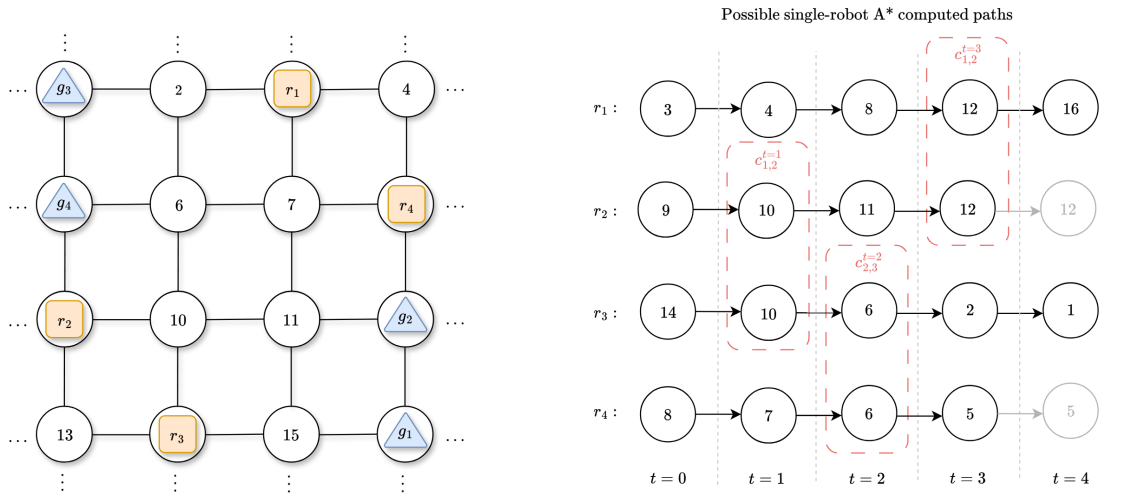
$$O(n) + O(n^2) + O(n) + O(n \log n). \tag{5.38}$$

5.6 Conflict-Based Heuristics

Conflict-based heuristics aim to estimate more explicitly the potential interference between robots. Unlike the previous heuristic families, which assign priorities based on individual distance, path length, topological properties, or local spatial context, this family uses information about interactions that may occur between the planned movements of different robots.

To obtain this information, an individual path is first computed for each robot while ignoring the remaining robots. These paths are then compared over time in order to identify predicted conflicts between pairs of robots. As a result, the priority assigned to each robot depends not only on its own path, but also on how that path interacts with the paths of the other robots. In this context, a conflict occurs when two robots are expected to require incompatible use of the same roadmap resource at the same time what may include vertex conflicts or edge conflicts.

To illustrate the conflict-based heuristics, Figure 5.7 combines the roadmap scenario and one possible set of independently computed single-robot A^* paths. Figure 5.7a shows the initial robot and goal configuration, while Figure 5.7b shows the corresponding predicted space-time conflicts are then used by the conflict-based heunflicts obtained when the individual paths are compared over time.



(a) Conflict scenario.

(b) Single-robot A^* paths and predicted conflicts.

Figure 5.7: Illustrative example for the conflict-based heuristics.

For each robot r_i, the associated conflict occurrences are represented by the set O_i. Each conflict occurrence contributes to the priority score through a coefficient α. The priority score is then defined as

$$score_i = \sum_{c \in O_i} \alpha^t_{R_c} \tag{5.39}$$

where R_c denotes the set of robots involved in conflict occurrence c, and t denotes the time step at which the conflict occurs.

Different conflict-based heuristics can be obtained by changing the definition of α. The underlying intuition is that robots involved in more relevant predicted conflicts are more likely to be difficult to coordinate in a prioritized planner such as TEA*. By assigning priorities according to conflict information, these heuristics attempt to plan first or last the robots whose individual paths are expected to interfere more strongly with the rest of the system.

5.6.1 Space-Time Conflicts

The Space-Time Conflicts heuristic corresponds to the simplest conflict-based variant. In this case, all predicted conflict occurrences are treated with the same importance. Therefore, each conflict occurrence has unit weight

$$\alpha^t_{R_c} = 1. \tag{5.40}$$

Substituting this definition into Equation 5.39, the priority score of robot r_i becomes

$$\text{score}(r_i) = |O_i|. \tag{5.41}$$

Thus, the score is simply the number of predicted space-time conflicts involving robot r_i. Robots involved in more conflicts receive higher priority scores, since their individual paths are expected to interfere more frequently with the paths of the remaining robots. For the example shown in Figure 5.7b, the predicted conflict occurrences are

$$c^{t=1}_{2,3}, \qquad c^{t=2}_{3,4}, \qquad c^{t=3}_{1,2}.$$

Since each conflict has unit weight, the resulting scores are

$$score_1 = 1, \qquad score_2 = 2, \qquad score_3 = 2, \qquad score_4 = 1.$$

Therefore, robots r_2 and r_3 obtain the highest conflict scores in this example, while robots r_1 and r_4 obtain lower scores. However, this score should be interpreted only as a static estimate of conflict involvement. The heuristic does not account for the fact that resolving an earlier conflict may change the subsequent paths and consequently remove conflicts that were originally predicted.

5.6.1.1 Ordering variants

Two ordering variants were considered for the *Space-Time Conflicts* heuristic. In the *More Conflicts First* variant, robots are sorted by decreasing score, so robots involved in more predicted conflicts are planned earlier. For the example in Figure 5.7b, this produces the priority grouping $\pi = \langle \{r_2, r_3\}, \{r_1, r_4\} \rangle$.

In the *Less Conflicts First* variant, the ordering is reversed, and robots with fewer predicted conflicts are planned earlier. The resulting priority grouping is $\pi = \langle \{r_1, r_4\}, \{r_2, r_3\} \rangle$.

5.6.1.2 Core Algorithm

Algorithm 9 summarizes the *Space-Time Conflicts* heuristic. The algorithm first computes independent single-robot paths and then counts the predicted space-time conflict occurrences involving each robot.

Algorithm 9 Space-Time Conflicts Heuristic

Require: Set of robots R, current positions s_i, target positions g_i, ordering mode *mode*
Ensure: Priority ordering π

for each robot $r_i \in R$ **do**
  $p_i \leftarrow A^*(s_i, g_i)$ ▷ single-robot shortest path on the roadmap
  $O_i \leftarrow \emptyset$ ▷ conflict occurrences involving robot r_i
end for
$L_{\max} \leftarrow \max_i |p_i|$ ▷ maximum individual path length
for $t = 0, 1, \ldots, L_{\max} - 1$ **do**
  for each pair of robots (r_i, r_j) with $i < j$ **do**
    if $t < |p_i|$ **and** $t < |p_j|$ **then**
      $e_i(t) \leftarrow$ edge traversed by robot r_i at timestep t
      $e_j(t) \leftarrow$ edge traversed by robot r_j at timestep t
      $V_{\text{conf}} \leftarrow$ shared endpoint vertices of $e_i(t)$ and $e_j(t)$
      if $V_{\text{conf}} \neq \emptyset$ **then**
        add conflict occurrence $(t, V_{\text{conf}}, r_j)$ to O_i
        add conflict occurrence $(t, V_{\text{conf}}, r_i)$ to O_j
      end if
    end if
  end for
end for
for each robot $r_i \in R$ **do**
  $score_i \leftarrow |O_i|$ ▷ number of predicted conflicts
end for
$\pi \leftarrow \textsc{SortByScore}(R, score, mode)$
return π

For each robot, the heuristic first computes a single-robot shortest path on the roadmap using A^*, and the resulting independent paths are then used to estimate the number of space-time conflicts involving each robot.

Using the graph-search complexity given in Equation 2.3, the path generation step is applied once per robot.

After the individual paths are obtained, the heuristic compares them over time. The maximum path length among all robots is denoted by $L_{\max} = \max_i |p_i|$. For each timestep, all unordered pairs

of robots are checked for conflicts. Since there are $O(n^2)$ robot pairs and each pairwise conflict check requires constant time, the conflict detection step costs

$$O(L_{\max} \cdot n^2). \tag{5.42}$$

The score of each robot is obtained by counting the conflict occurrences stored in O_i. This operation is bounded by the number of detected conflicts and, in the worst case, does not exceed the pairwise conflict detection cost. The final score-based sorting step requires $O(n \log n)$ time. Thus, the total computational cost is given by

$$O\big(n \cdot (|V| + |E|) \log |V|\big) + O\big(L_{\max} \cdot n^2\big) + O(n \log n). \tag{5.43}$$

5.6.2 Time-DoF Weighted Conflicts

The *Time-DoF Weighted Conflicts* heuristic extends the previous conflict-based formulation by assigning a different weight to each predicted conflict occurrence. The objective is to distinguish conflicts not only by how often they occur, but also by when and where they occur along the individually planned paths.

For each conflict occurrence $c \in O_i$, the timestep at which the conflict occurs is denoted by t_c, the set of robots involved is denoted by $\mathscr{R}_c$, and the set of vertices associated with the conflict is denoted by V_c. The coefficient assigned to occurrence c is defined as

$$\alpha_{R_c}^{t_c} = \frac{1}{\min\limits_{v \in V_c} \mathrm{DoF}(v)} \cdot \frac{1}{1 + t_c}. \tag{5.44}$$

, where V_c contains the vertices associated with the conflict occurrence. The term $\min_{v \in V_c} \mathrm{DoF}(v)$ makes the coefficient depend on the most constrained vertex involved in the conflict. For vertex conflicts, this corresponds to the shared vertex. For edge interactions, V_c includes the vertices associated with the interacting edges, and the minimum DoF is used to represent the most constrained part of that interaction.

The temporal term gives higher weight to conflicts that occur earlier in the individual paths, based on the assumption that the first steps are more likely to be executed as originally computed. As time progresses, reservations introduced by other robots may force path changes, making later predicted conflicts less reliable indicators of actual future interference. This partially addresses a limitation of the previous heuristic, where all conflicts were counted equally, even though resolving an earlier conflict may alter subsequent paths and remove conflicts that were initially detected.

The DoF term gives higher weight to conflicts occurring in more constrained roadmap regions. When a conflict involves vertices with low DoF, the affected robots have fewer alternative movements available around that location. Such conflicts are therefore considered more critical than conflicts occurring in highly connected regions, where robots may have more possible escape routes or alternative paths.

Substituting the weight of Equation 5.44 into the general conflict-based score defined in Equation 5.39, the score of robot r_i becomes

$$score_i = \sum_{c \in O_i} \left(\frac{1}{\min\limits_{v \in V_c} \mathrm{DoF}(v)} \cdot \frac{1}{1+t_c} \right). \tag{5.45}$$

For the example shown in Figure 5.7b, assume that all vertices involved in the predicted conflicts have $\mathrm{DoF}(v) = 4$. Under this assumption, the coefficients are determined only by the timestep of each conflict:

$$\alpha_{2,3}^{t=1} = \frac{1}{4} \cdot \frac{1}{1+1} = \frac{1}{8}, \qquad \alpha_{3,4}^{t=2} = \frac{1}{4} \cdot \frac{1}{1+2} = \frac{1}{12}, \qquad \alpha_{1,2}^{t=3} = \frac{1}{4} \cdot \frac{1}{1+3} = \frac{1}{16}.$$

The resulting weighted scores are

$$score_1 = \frac{1}{16}, \qquad score_2 = \frac{1}{8} + \frac{1}{16} = \frac{3}{16}, \qquad score_3 = \frac{1}{8} + \frac{1}{12} = \frac{5}{24}, \qquad score_4 = \frac{1}{12}.$$

5.6.2.1 Ordering variants

Two ordering variants were considered. In the *High Time-DoF Weighted Conflicts First* variant, robots with higher weighted conflict scores are planned earlier. This prioritizes robots involved in conflicts that occur earlier and in more constrained roadmap regions. For the example shown in Figure 5.7b, this produces the ordering $\pi = \langle r_3, r_2, r_4, r_1 \rangle$.

Figure 5.8 shows a possible conflict-free set of paths generated by TEA* using this ordering. In this example, robot r_2 waits at timestep $t = 1$, while robot r_4 waits at timestep $t = 2$. These waiting actions resolve the earlier predicted conflicts. As a consequence, the later predicted conflict between r_1 and r_2 at timestep $t = 3$, visible in the independently computed paths of Figure 5.7b, no longer occurs in the final TEA* plan.

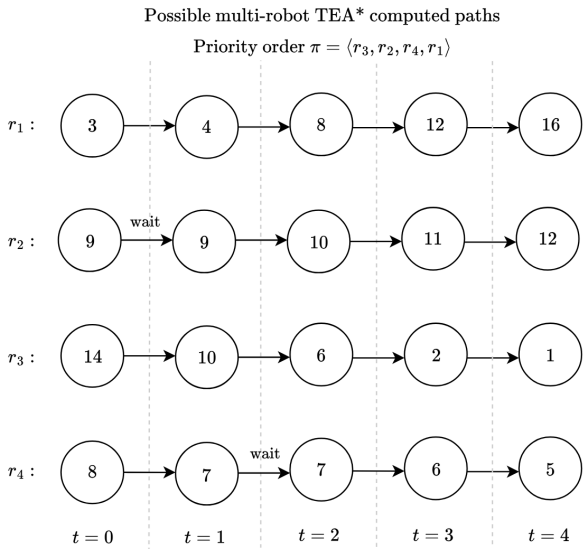


Figure 5.8: Possible conflict-free TEA* plan.

This illustrates the role of the temporal weighting term, which reduces the influence of later conflicts that may disappear after earlier interactions are resolved. In the *Low Time-DoF Weighted Conflicts First* variant, the ordering is reversed, with lower weighted conflict scores planned earlier.

5.6.2.2 Core Algorithm

Algorithm 10 follows the same conflict-detection structure as Algorithm 9, differing only in the weighted score computation.

Algorithm 10 Time-DoF Weighted Conflicts Heuristic

Require: Set of robots R, current positions s_i, target positions g_i, ordering mode *mode*
Ensure: Priority ordering π

for each robot $r_i \in R$ **do**
  $p_i \leftarrow A^*(s_i, g_i)$ ▷ single-robot shortest path on the roadmap
  $O_i \leftarrow \emptyset$ ▷ conflict occurrences involving robot r_i
end for
$L_{\max} \leftarrow \max_i |p_i|$ ▷ maximum individual path length
for $t = 0, 1, \ldots, L_{\max} - 1$ **do**
  for each pair of robots (r_i, r_j) with $i < j$ **do**
    if $t < |p_i|$ **and** $t < |p_j|$ **then**
      $e_i(t) \leftarrow$ edge traversed by robot r_i at timestep t
      $e_j(t) \leftarrow$ edge traversed by robot r_j at timestep t
      $V_{\text{conf}} \leftarrow$ shared endpoint vertices of $e_i(t)$ and $e_j(t)$
      if $V_{\text{conf}} \neq \emptyset$ **then**
        add conflict occurrence $(t, V_{\text{conf}}, r_j)$ to O_i
        add conflict occurrence $(t, V_{\text{conf}}, r_i)$ to O_j
      end if
    end if
  end for
end for
for each robot $r_i \in R$ **do**
  $score_i \leftarrow 0$
  for each occurrence $(t, V_{\text{conf}}, r_j) \in O_i$ **do**
    $DoF_{\min} \leftarrow \min_{v \in V_{\text{conf}}} \text{DoF}(v)$ ▷ most constrained associated vertex
    $\alpha \leftarrow \frac{1}{DoF_{\min}} \cdot \frac{1}{1+t}$ ▷ Time-DoF conflict coefficient
    $score_i \leftarrow score_i + \alpha$
  end for
end for
$\pi \leftarrow \text{SORTBYSCORE}(R, score, mode)$
return π

The computational complexity is approximately the same as that of the unweighted *Space-Time Conflicts* heuristic, given in Equation 5.43. The weighted variant only adds a constant-time weight computation for each detected conflict occurrence, whose impact can be considered negligible.

5.7 Replanning Heuristics

Replanning heuristics differ from the previous heuristic families because they are not intended to compute the initial priority ordering from static information alone. Instead, they use information collected during previous planning attempts in order to guide subsequent reordering decisions. These heuristics are therefore only applicable when the planner has already produced search statistics that can be used to estimate which robots were more affected by the current priority ordering.

In this work, one replanning heuristic was developed, named *Exploration Conflicts Pressure*. This heuristic uses reservation-conflict statistics collected during a previous TEA* search attempt to compute a revised robot ordering for a subsequent planning attempt.

5.7.1 Exploration Conflicts Pressure

The *Exploration Conflicts Pressure* heuristic is designed for replanning scenarios, especially when a previous TEA* attempt fails or produces a poor-quality solution. Unlike the initial-ordering heuristics, this heuristic does not estimate difficulty from distances, topology, local density, or predicted conflicts between independently planned paths. Instead, it uses runtime feedback collected during the actual prioritized planning process to revise the robot ordering.

The main idea is to identify robots whose search was strongly constrained by reservations introduced by previously planned robots. During TEA*, a candidate expansion may be rejected if the corresponding vertex or edge is already reserved in space-time by a higher-priority robot. If this happens frequently during the search of robot r_i, it suggests that the robot was negatively affected by its position in the previous ordering. Rather than simply reversing the previous permutation, the heuristic performs a targeted reordering, moving earlier only the robots that show evidence of being constrained by existing reservations.

For robot r_i, N_i^{rej} denotes the number of reservation-based rejections observed during the search, and E_i denotes the total number of expanded nodes. Instead of using N_i^{rej} directly, the heuristic normalizes this value by the total exploration effort E_i. This avoids assigning high scores only because a robot required a longer or more extensive search, which naturally creates more opportunities for reservation conflicts to occur. The priority score assigned to robot r_i is therefore defined as

$$score_i = \begin{cases} \dfrac{N_i^{\text{rej}}}{E_i}, & \text{if } E_i > 0, \\ 0, & \text{otherwise.} \end{cases} \tag{5.46}$$

This score measures the fraction of the search effort affected by reservation conflicts. Higher values indicate that a larger proportion of the attempted exploration was blocked by existing space-time reservations, rather than simply indicating that the robot performed a larger search. Therefore, robots with higher scores are interpreted as being more constrained by the previous ordering and may benefit from receiving higher priority in a subsequent planning attempt.

Figure 5.9 illustrates the type of runtime information used by this heuristic. In the represented TEA* expansion step, robot r_2 is being planned after other robots have already introduced space-time reservations. Three possible successor expansions are considered. Two of them are rejected because the corresponding roadmap resources are already reserved by previously planned robots, while only one expansion remains feasible. Therefore, this step contributes two reservation-based rejections to the value of R_2.

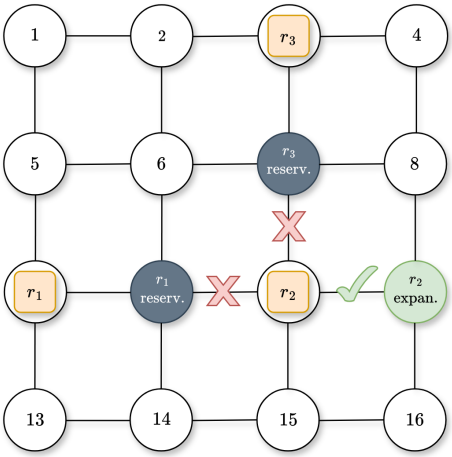


Figure 5.9: TEA* expansion step illustrating two reservation-based rejections.

This example shows why the ratio in Equation 5.46 can be useful during replanning. If similar rejections occur frequently throughout the search of a robot, this indicates that the robot was strongly affected by reservations imposed by higher-priority robots. The heuristic then uses this information to adjust the ordering in a subsequent planning attempt.

5.7.1.1 Core Algorithm

Algorithm 11 summarizes the *Exploration Conflicts Pressure* heuristic. This heuristic is applied only in a retry context and uses search statistics collected during a previous TEA* planning attempt. The statistics used by the heuristic are extracted from the previous TEA* planning attempt. The value E_i corresponds to the number of space-time nodes expanded while planning robot r_i, while N_i^{rej} corresponds to the number of candidate expansions rejected because the required vertex or edge was already reserved by a higher-priority robot.

Robots with higher scores encountered reservation conflicts more frequently during their search. In the *High Exploration Conflicts Pressure First* variant, these robots are moved earlier in the ordering, with the objective of reducing the blocking effect observed in the previous attempt. In the

opposite variant, *Low Exploration Conflicts Pressure First*, robots with lower scores are planned earlier.

No additional individual path computation or pairwise conflict enumeration is performed by this heuristic. The reordering relies only on statistics already collected during a previous TEA* execution.

Algorithm 11 Exploration Conflicts Pressure Heuristic

Require: Planning queue Q, search statistics S from the previous planning attempt, ordering mode *mode*

Ensure: Revised priority ordering π

for each robot $r_i \in Q$ **do**
  if $r_i \notin S$ **or** $S[r_i].E_i = 0$ **then**
    $score_i \leftarrow 0$
  else
    $N_i^{\text{rej}} \leftarrow S[r_i].N_i^{\text{rej}}$ ▷ reservation-based rejections
    $E_i \leftarrow S[r_i].E_i$ ▷ expanded nodes
    $score_i \leftarrow \dfrac{N_i^{\text{rej}}}{E_i}$
  end if
end for
$\pi \leftarrow \text{SORTBYSCORE}(Q, score, mode)$
return π

The number of robots in the planning queue is denoted by $n = |Q|$. For each robot, the heuristic retrieves the corresponding search statistics and computes the ratio between reservation-based rejections and expanded nodes. Assuming that the statistics are accessed in constant time, the score computation step requires $O(n)$ time.

The final score-based sorting step requires $O(n \log n)$ time. Thus, the total computational cost is given by

$$O(n) + O(n \log n). \tag{5.47}$$

The cost of this heuristic is small compared with the TEA* planning attempt that produced the statistics. Its practical overhead is therefore limited, since it only reorders the robots using information already collected during the previous search.

5.8 Summary

This chapter presented the prioritization heuristics developed to generate alternative robot orderings for the TEA* planner. Since TEA* plans robots sequentially, the order in which robots are planned directly affects the reservations introduced in the space-time search space and, consequently, the feasibility and quality of the resulting solution.

The proposed heuristics were organized into different families according to the information used to compute the priority scores. Distance-based heuristics estimate the movement effort of each robot using either Euclidean distance or roadmap path cost. Topological heuristics evaluate the degree of movement flexibility along each robot path through DoF-based metrics. Density-based heuristics analyze the spatial distribution of robots using either geometric neighbourhoods, roadmap-based k-hop neighbourhoods, or cluster-based structures. Conflict-based heuristics rely on predicted interactions between independently planned paths, while replanning heuristics exploit information collected from previous TEA* attempts.

Each heuristic family therefore represents a different search criterion for selecting promising priority orderings. Since no single criterion is expected to be consistently optimal across all scenarios, these complementary heuristics motivate the use of a parallel exploration strategy, where different threads can evaluate different priority orderings simultaneously.

The chapter also analyzed the computational cost of each heuristic. Simpler heuristics, such as Euclidean distance or retry-based score computation, introduce low overhead, while heuristics that require graph searches, clustering, neighbourhood expansion, or pairwise conflict detection are computationally more expensive. These analyses provide the basis for evaluating the trade-off between heuristic informativeness and computational cost.

The heuristics described in this chapter define the set of candidate priority orderings explored in the next chapter, where they are executed within the proposed parallel coordination framework.

Chapter 6

Parallelization and Heuristic Dispatching Framework

Chapter 7

Experimental Evaluation and Performance Analysis

Chapter 8

Conclusion

This document established the foundations and technical direction for addressing centralized multi-robot coordination under the MAPF formulation. Through the analysis of MAPF fundamentals and state-of-the-art coordination paradigms, it became clear that centralized approaches remain particularly well suited for scenarios requiring strong safety, predictability, and global consistency guarantees, despite their inherent scalability challenges.

The detailed study of centralized planning methods highlighted the advantages of TEA* in practical fleet coordination systems, namely its explicit handling of temporal constraints and its suitability for online replanning. However, the sequential and prioritized nature of TEA* was identified as a key limitation, as planning outcomes are highly sensitive to agent ordering and no single prioritization heuristic performs consistently well across different environments and task configurations.

Motivated by this observation, a preliminary solution was proposed that extends the existing TEA*-based planner through parallel exploration of multiple priority orderings. By executing several sequential TEA* instances concurrently and selecting the best resulting plan, the proposed approach aims to improve robustness and solution quality without altering the core architecture of the centralized coordination framework. The discussion of alternative execution and synchronization strategies further emphasized the potential to balance planning latency and exploration depth through tunable parameters.

Overall, the conclusions drawn in this document motivate the dissertation phase, which will focus on validating whether parallel prioritization can effectively mitigate ordering sensitivity in TEA*-based fleet coordination systems under realistic conditions.

References

[1] R. Stern, N. R. Sturtevant, A. Felner, S. Koenig, H. Ma, T. T. Walker, J. Li, D. Atzmon, L. Cohen, T. K. S. Kumar, E. Boyarski, and R. Barták, "Multi-agent pathfinding: Definitions, variants, and benchmarks," *Proceedings of the 12th International Symposium on Combinatorial Search, SoCS 2019*, pp. 151–158, Jun. 2019, ISSN: 2832-9171. DOI: 10.1609/socs.v10i1.18510.

[2] J. Yu and S. M. Lavalle, "Planning optimal paths for multiple robots on graphs," *Proceedings - IEEE International Conference on Robotics and Automation*, pp. 3612–3617, Apr. 2012, ISSN: 10504729. DOI: 10.1109/ICRA.2013.6631084.

[3] G. Sharon, R. Stern, A. Felner, and N. R. Sturtevant, "Conflict-based search for optimal multi-agent pathfinding," *Artificial Intelligence*, vol. 219, pp. 40–66, Feb. 2015, ISSN: 0004-3702. DOI: 10.1016/J.ARTINT.2014.11.006.

[4] D. M. Matos, P. Costa, H. Sobreira, A. Valente, and J. Lima, "Efficient multi-robot path planning in real environments: A centralized coordination system," *International Journal of Intelligent Robotics and Applications*, vol. 9, pp. 217–244, 1 Mar. 2025, ISSN: 2366598X. DOI: 10.1007/s41315-024-00378-3.

[5] D. Silver, "Cooperative pathfinding," *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, vol. 1, pp. 117–122, 1 Jun. 2005, ISSN: 2334-0924. DOI: 10.1609/AIIDE.V1I1.18726.

[6] P. Moura, P. Costa, J. Lima, and P. Costa, "A temporal optimization applied to time enhanced a," vol. 2116, American Institute of Physics Inc., Jul. 2019, ISBN: 9780735418547. DOI: 10.1063/1.5114225.

[7] P. Surynek, "An optimization variant of multi-robot path planning is intractable," vol. 24, AAAI Press, Jul. 2010, pp. 1261–1263, ISBN: 9781577354642. DOI: 10.1609/AAAI.V24I1.7767.

[8] T. Geft, "Fine-grained complexity analysis of multi-agent path finding on 2d grids," *The International Symposium on Combinatorial Search*, vol. 16, pp. 20–28, 1 May 2023, ISSN: 28329163. DOI: 10.1609/socs.v16i1.27279.

[9] J. Gao, Y. Li, X. Li, K. Yan, K. Lin, and X. Wu, "A review of graph-based multi-agent pathfinding solvers," *Knowledge-Based Systems*, vol. 283, Jan. 2024, ISSN: 09507051. DOI: 10.1016/J.KNOSYS.2023.111121.

[10] A. Yang, Q. Niu, W. Zhao, K. Li, and G. W. Irwin, "An efficient algorithm for grid-based robotic path planning based on priority sorting of direction vectors," vol. 6329 LNCS, Springer, Berlin, Heidelberg, 2010, pp. 456–466, ISBN: 978-3-642-15597-0. DOI: 10.1007/978-3-642-15597-0_50.

[11] H. Ma, "Graph-based multi-robot path finding and planning," *Current Robotics Reports*, vol. 3, pp. 77–84, 3 Jun. 2022. DOI: 10.1007/s43154-022-00083-8.

[12] T. Lozano-Pérez and M. A. Wesley, "An algorithm for planning collision-free paths among polyhedral obstacles," *Communications of the ACM*, vol. 22, pp. 560–570, 10 Oct. 1979, ISSN: 15577317. DOI: 10.1145/359156.359164.

[13] F. Aurenhammer, "Voronoi diagrams—a survey of a fundamental geometric data structure," *ACM Computing Surveys (CSUR)*, vol. 23, pp. 345–405, 3 Jan. 1991, ISSN: 15577341. DOI: 10.1145/116873.116880.

[14] J.-C. Latombe, *Robot Motion Planning*. Springer, 1991.

[15] K. L. Seth Hutchinson Howie Choset, *Principles of Robot Motion*. 2005, vol. 53, pp. 1079–1083, ISBN: 9780262033275. [Online]. Available: https://mitpress.mit.edu/9780262033275/principles-of-robot-motion/.

[16] S. J. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 3rd ed. Prentice Hall, 2010, ISBN: 978-0136042594.

[17] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA: MIT Press, Sep. 2009, ISBN: 9780262033848. [Online]. Available: https://mitpress.mit.edu/9780262033848/introduction-to-algorithms/.

[18] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, pp. 100–107, 2 1968, ISSN: 21682887. DOI: 10.1109/TSSC.1968.300136.

[19] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, pp. 269–271, 1 Dec. 1959, ISSN: 0029599X. DOI: 10.1007/BF01386390.

[20] T. L. Rankin, *Heuristics: Intelligent Search Strategies for Computer Problem Solving. Judea Pearl. Addison-Wesley Publishing. 1984. 382 pp.* 1986, vol. 7, pp. 87–89. DOI: https://doi.org/10.1609/aimag.v7i1.534.

[21] R. H. Arpaci-Dusseau and A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*. Arpaci-Dusseau Books, 2018. [Online]. Available: https://pages.cs.wisc.edu/~remzi/OSTEP/.

[22] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Wiley, 2018, ISBN: 978-1119456339.

[23] A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, 4th ed. Pearson, 2015, ISBN: 978-0133591620.

[24] E. W. Dijkstra, "Cooperating sequential processes," pp. 43–112, 1968.

[25] L. Siefke, V. Sommer, B. Wudka, C. Thomas, L. Siefke, V. Sommer, B. Wudka, and C. Thomas, "Robotic systems of systems based on a decentralized service-oriented architecture," *Robotics 2020, Vol. 9,*, vol. 9, pp. 1–10, 4 Sep. 2020, ISSN: 2218-6581. DOI: 10.3390/ROBOTICS9040078.

[26] P. Flocchini, G. Prencipe, N. Santoro, and P. Widmayer, "Distributed coordination of a set of autonomous mobile robots," *IEEE Intelligent Vehicles Symposium, Proceedings*, pp. 480–485, 2000. DOI: 10.1109/IVS.2000.898389.

[27] M. Čáp, P. Novák, J. Vokřínek, and M. Pěchouček, *Asynchronous Decentralized Algorithm for Space-Time Cooperative Pathfinding*. Oct. 2012. [Online]. Available: https://arxiv.org/abs/1210.6855v1.

[28] J. D. V. Berg, M. Lin, and D. Manocha, "Reciprocal velocity obstacles for real-time multi-agent navigation," *Proceedings - IEEE International Conference on Robotics and Automation*, pp. 1928–1935, 2008, ISSN: 10504729. DOI: 10.1109/ROBOT.2008.4543489.

[29] O. Khatib, "Real-time obstacle avoidance for manipulators and mobile robots," *The International Journal of Robotics Research*, vol. 5, pp. 90–98, 1 1986, ISSN: 02783649. DOI: 10.1177/027836498600500106.

[30] Z. Wang, X. Zhao, J. Zhang, N. Yang, P. Wang, J. Tang, J. Zhang, and L. Shi, "Apf-cpp: An artificial potential field based multi-robot online coverage path planning approach," *IEEE Robotics and Automation Letters*, vol. 9, pp. 9199–9206, 11 2024, ISSN: 23773766. DOI: 10.1109/LRA.2024.3432351.

[31] Z. Wu, J. Dai, B. Jiang, and H. R. Karimi, "Robot path planning based on artificial potential field with deterministic annealing," *ISA Transactions*, vol. 138, pp. 74–87, Jul. 2023, ISSN: 00190578. DOI: 10.1016/J.ISATRA.2023.02.018.

[32] P. Caloud, W. Choi, J. C. Latombe, C. L. Pape, and M. Yim, "Indoor automation with many mobile robots," *IEEE International Conference on Intelligent Robots and Systems*, vol. 1990-July, pp. 67–72, 1990, ISSN: 21530866. DOI: 10.1109/IROS.1990.262370.

[33] J. Santos, P. Costa, L. F. Rocha, A. P. Moreira, and G. Veiga, "Time enhanced a: Towards the development of a new approach for multi-robot coordination," *Proceedings of the IEEE International Conference on Industrial Technology*, vol. 2015-June, pp. 3314–3319, June Jun. 2015. DOI: 10.1109/ICIT.2015.7125589.

[34] J. Santos, P. Costa, L. Rocha, K. Vivaldini, A. P. Moreira, and G. Veiga, "Validation of a time based routing algorithm using a realistic automatic warehouse scenario," *Advances in Intelligent Systems and Computing*, vol. 418, pp. 81–92, 2016, ISSN: 2194-5365. DOI: 10.1007/978-3-319-27149-1_7.

[35] A. Andreychuk, K. Yakovlev, E. Boyarski, and R. Stern, "Improving continuous-time conflict based search," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, pp. 11 220–11 227, 13 May 2021, ISSN: 2374-3468. DOI: 10.1609/AAAI.V35I13.17338.

[36] M. Phillips and M. Likhachev, "Sipp: Safe interval path planning for dynamic environments," *Proceedings - IEEE International Conference on Robotics and Automation*, pp. 5628–5635, 2011, ISSN: 10504729. DOI: 10.1109/ICRA.2011.5980306.

[37] M. Barer, G. Sharon, R. Stern, and A. Felner, "Suboptimal variants of the conflict-based search algorithm for the multi-agent pathfinding problem," *Proceedings of the International Symposium on Combinatorial Search*, vol. 5, pp. 19–27, 1 2014, ISSN: 2832-9163. DOI: 10.1609/SOCS.V5I1.18315.

[38] A. Felner, J. Li, E. Boyarski, H. Ma, L. Cohen, T. K. Kumar, and S. Koenig, "Adding heuristics to conflict-based search for multi-agent path finding," *Proceedings of the International Conference on Automated Planning and Scheduling*, vol. 28, pp. 83–87, Jun. 2018, ISSN: 2334-0843. DOI: 10.1609/ICAPS.V28I1.13883.

[39] J. Li, D. Harabor, P. J. Stuckey, H. Ma, G. Gange, and S. Koenig, "Pairwise symmetry reasoning for multi-agent path finding search," *Artificial Intelligence*, vol. 301, p. 103 574, 2021, ISSN: 0004-3702. DOI: https://doi.org/10.1016/j.artint.2021.103574.

[40] B. Muppasani, R. Dey, B. Srivastava, and V. Narayanan, "Scalable multi-agent path finding using collision-aware dynamic alert mask and a hybrid execution strategy," Oct. 2025. [Online]. Available: https://arxiv.org/abs/2510.09469v1.

[41] T. Standley, "Finding optimal solutions to cooperative pathfinding problems," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 24, pp. 173–178, 1 Jul. 2010, ISSN: 2374-3468. DOI: 10.1609/AAAI.V24I1.7564.

[42] H. Zhang, M. Yao, Z. Liu, J. Li, L. Terr, S. H. Chan, T. K. S. Kumar, and S. Koenig, "A hierarchical approach to multi-agent path finding," *Proceedings of the International Symposium on Combinatorial Search*, vol. 12, pp. 209–211, 1 Jul. 2021, ISSN: 2832-9163. DOI: 10.1609/SOCS.V12I1.18586.

[43] C. Leet, J. Li, and S. Koenig, "Shard systems: Scalable, robust and persistent multi-agent path finding with performance guarantees," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 36, pp. 9386–9395, 9 Jun. 2022, ISSN: 2374-3468. DOI: 10.1609/AAAI.V36I9.21170.

[44] S. Wang, H. Xu, Y. Zhang, J. Lin, C. Lu, X. Wang, and W. Li, "Where paths collide: A comprehensive survey of classic and learning-based multi-agent pathfinding," May 2025. [Online]. Available: https://arxiv.org/abs/2505.19219v2.

[45] J. Hu, "A survey on reinforcement learning-based multi-agent path planning," *ITM Web of Conferences*, vol. 78, Jan. 2025. DOI: 10.1051/itmconf/20257801015.

[46] M. Everett, Y. F. Chen, and J. P. How, "Collision avoidance in pedestrian-rich environments with deep reinforcement learning," *IEEE Access*, vol. 9, pp. 10 357–10 377, 2021. DOI: 10.1109/ACCESS.2021.3050338.

[47] J. R. Heselden, G. P. Das, J. R. Heselden, and G. P. Das, "Heuristics and rescheduling in prioritised multi-robot path planning: A literature review," *Machines 2023, Vol. 11,*, vol. 11, 11 Nov. 2023, ISSN: 2075-1702. DOI: 10.3390/MACHINES11111033.